UNIVERSIDAD NACIONAL DEL ALTIPLANO

Escuela Profesional de Ingeniería de Sistemas




# AzChat

Documentación del sistema de chat universitario

Planificación, diseño y construcción

**Curso:** Sistemas Distribuidos

**Docente:** SUCASACA PACORI AMILKAR

**Integrantes**

Apaza Quispe Antonio Vicente · 236876

Arias Ramirez Alex · 236104

Chura Chambilla Raul Cristian · 236567

Escobedo Arapa Kenyi Blanca · 236171

Mamani Pari Yimmy Jonathan · 237317

Quispe Ramos Daniel Jhosimar · 236568

Roque Mamani Vidman Ruis · 236232

Vidal Fuentes Jordy Anthony · 236535

Puno, Perú · 2026

# Guía de lectura

Facilitar la intercomunicación entre estudiantes de la Universidad Nacional del Altiplano mediante un sistema de chat universitario.

Los pasos 1–11 reúnen el análisis y el diseño; el paso 12 explica la construcción por incrementos, las revisiones y la evidencia de validación. Revisión documental: 6 de noviembre de 2026.

**P1** La idea y sus límites

**P2** Las personas que lo usarán

**P3** Funciones del sistema

**P4** Las acciones de cada persona

**P5** Las reglas del chat

**P6** Los elementos del sistema

**P7** El recorrido de un mensaje

**P8** Cómo se reparte el trabajo

**P9** Cómo se conectan sus partes

**P10** Cómo se usa AzChat

**P11** La información que se conserva

**P12** Construcción y validación

Aplicación web de AzChat · Documentación y modelos editables

## Metodología de desarrollo

Para la construcción de **AzChat** se empleó un modelo de desarrollo incremental e iterativo, con un enfoque adaptativo.

El proyecto comenzó con una planificación inicial que permitió definir las funcionalidades principales del sistema y desarrollar una primera versión funcional.

Posteriormente, durante el proceso de desarrollo, se identificaron nuevas necesidades y oportunidades de mejora. Cada una de ellas fue analizada y planificada antes de proceder con su implementación e integración al sistema existente.

Este procedimiento permitió ampliar progresivamente las capacidades de AzChat, adaptando el desarrollo a los nuevos requisitos sin necesidad de reconstruir completamente la aplicación.

### Alcance de la lectura

Los bocetos muestran una revisión esquemática del diseño. P12 distingue lo construido de las validaciones pendientes. El formato del código estudiantil se comprueba, pero no se acredita matrícula contra un padrón oficial. La documentación no contiene credenciales de la aplicación.

# P1 · La idea y sus límites

La necesidad que atiende el proyecto y sus límites.

## Objetivo general

**Facilitar la intercomunicación entre estudiantes de la Universidad Nacional del Altiplano mediante un sistema de chat universitario** que permita conversar, coordinar actividades académicas y compartir información desde una computadora o un celular.

## La necesidad que atenderá

Los estudiantes necesitan comunicarse con compañeros de su universidad para resolver dudas, coordinar trabajos y mantener el contexto de sus intercambios. AzChat propone reunir esas conversaciones en una cuenta personal, con espacios privados y de grupo e historial consultable.

## Objetivos específicos

- Permitir que un estudiante encuentre a otro y establezca una conversación privada.
- Ofrecer grupos para coordinar equipos, cursos o actividades entre estudiantes.
- Compartir mensajes y materiales que puedan retomarse desde el historial.
- Proteger el acceso a cada conversación y permitir recuperar la cuenta de forma controlada.

## Alcance y límites

El público objetivo son los estudiantes de la universidad. El código de estudiante identifica la cuenta, pero el diseño no incluye una consulta al padrón universitario para acreditar matrícula. No se presupone integración con sistemas académicos oficiales.

Se contemplan las funciones del catálogo de P3. Quedan fuera las llamadas de voz o video y el cifrado de extremo a extremo: el servidor necesita procesar y almacenar el contenido. Las condiciones para evaluar el diseño se reúnen en P12.

# P2 · Las personas que lo usarán

Quién participa y qué necesita hacer.

## Personas y responsabilidades

| Participante | Necesidad y responsabilidad |
|---|---|
| Estudiante de la universidad | Intercomunicarse con otros estudiantes y gestionar su propia cuenta. Accede a las conversaciones de las que forma parte. |
| Administrador de un grupo estudiantil | Organizar sus participantes, roles y datos. Su permiso se limita al grupo que administra. |
| Responsable de recuperación | Revisar la identidad del solicitante cuando se propone un correo nuevo sin poder ingresar. La aprobación permite verificar el correo; no cambia la contraseña. |

## Necesidades en un contexto universitario

Dos compañeros coordinan una entrega por chat privado. Un equipo comparte avances en un grupo y añade a otro estudiante cuando se incorpora al trabajo. Si alguien pierde el acceso, debe recuperar su cuenta conservando el historial y sin abrirlo a terceros.

## Servicios de apoyo

El alojamiento ejecuta la aplicación; PostgreSQL conserva los intercambios; el proveedor de correo entrega códigos y solicitudes; el servicio Web Push transporta avisos a dispositivos autorizados. Estos servicios son apoyos técnicos, no perfiles con permiso para administrar grupos.

# P3 · Funciones del sistema

Catálogo de funciones previstas, organizado por objetivo.

## Catálogo funcional

Las funciones se organizan para cumplir el fin de AzChat: que los estudiantes de la universidad puedan intercomunicarse y coordinar sus actividades. Las reglas se consultan en P5 y las acciones por participante en P4.

| ID | Función prevista |
|---|---|
| RF-01 | Crear una cuenta estudiantil con correo de recuperación opcional. |
| RF-02 | Ingresar, cerrar sesión y cambiar la contraseña. |
| RF-03 | Editar foto, nombre y descripción del perfil. |
| RF-04 | Buscar compañeros por nombre o usuario y abrir un chat privado único por pareja. |
| RF-05 | Crear grupos estudiantiles y administrar sus participantes y datos. |

| ID | Función prevista |
|---|---|
| RF-06 | Enviar texto, responder, editar y eliminar mensajes propios. |
| RF-07 | Compartir archivos e imágenes con previsualización y edición previa. |
| RF-08 | Grabar y reproducir notas de voz. |
| RF-09 | Consultar y buscar historial, ver lectura y reintentar envíos fallidos. |
| RF-10 | Fijar o silenciar conversaciones y exportar historial a TXT. |
| RF-11 | Elegir apariencia y preferencias; activar avisos e instalar la web si el dispositivo lo permite. |
| RF-12 | Vincular un correo verificado y recuperar el acceso con la autorización correspondiente. |
| RF-13 | Elegir un nombre privado para un compañero, visible solo para quien lo guarda. |
| RF-14 | Consultar indicaciones contextuales mediante el botón !. |

## Condiciones de calidad

| ID | Condición |
|---|---|
| RNF-01 | Autorización por sesión y pertenencia; una cuenta ajena no accede al chat ni a sus adjuntos. |
| RNF-02 | Conservación del historial confirmado y tratamiento de reintentos sin duplicar mensajes. |
| RNF-03 | Uso con teclado, controles identificables y navegación adaptable al celular. |
| RNF-04 | Credenciales privadas en el servidor y conexión protegida con la base de datos. |
| RNF-05 | Capacidad y tiempo de respuesta medidos con el escenario de aceptación de P12. |

# P4 · Las acciones de cada persona

Cómo se relacionan las personas con la aplicación.

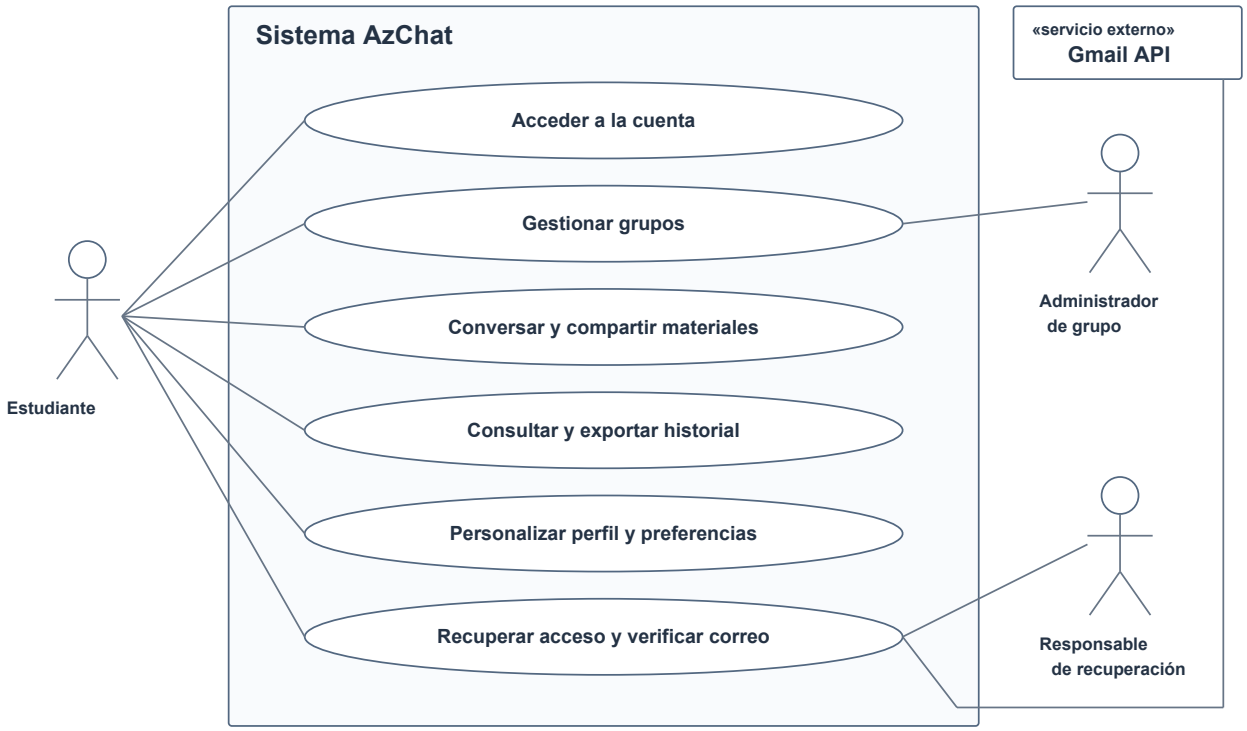
## CU-00 Caso de uso general de AzChat

**Objetivo:** intercomunicarse con otros estudiantes y conservar los intercambios de una coordinación académica en una cuenta personal.

**Actor principal:** estudiante. **Actores de apoyo:** otros participantes, administrador del grupo y responsable de recuperación. Gmail API es el servicio externo de correo.

## CU-00 · Uso general de AzChat

AzChat · Diseño revisado · Elementos simplificados y datos ficticios




**Participación por objetivo: las opciones no son pasos obligatorios de una misma sesión.**

Vista general de objetivos. Las líneas representan participación; no indican un orden obligatorio de ejecución.

| Elemento | Descripción |
|---|---|
| Precondición | Disponer de un navegador y conexión. Para acceder a conversaciones se requiere una cuenta y una sesión válida. |
| Inicio | El estudiante desea comunicarse con un compañero o equipo. |
| Flujo principal | 1. Crear la cuenta o ingresar. 2. Buscar a un compañero o elegir un grupo. 3. Intercambiar texto, imágenes, archivos o notas de voz. 4. Consultar el historial y las respuestas. 5. Ajustar perfil o preferencias si lo necesita. 6. Cerrar la sesión al terminar. |
| Alternativas | Si olvidó la contraseña, iniciar recuperación. Si falla un envío, revisar la conexión y reintentar. Si no pertenece a una conversación, el sistema deniega su acceso. Crear grupos, exportar chats y activar avisos son opciones según la necesidad. |
| Resultado | El intercambio queda disponible para sus participantes autorizados. El estudiante puede volver a consultarlo al ingresar nuevamente. |
| Relación con el detalle | CU-01 a CU-13 desarrollan los recorridos específicos. CU-00 sirve como mapa de lectura y no añade un requisito independiente ni obliga a ejecutar todas las funciones en cada sesión. |

## Comunicación entre estudiantes

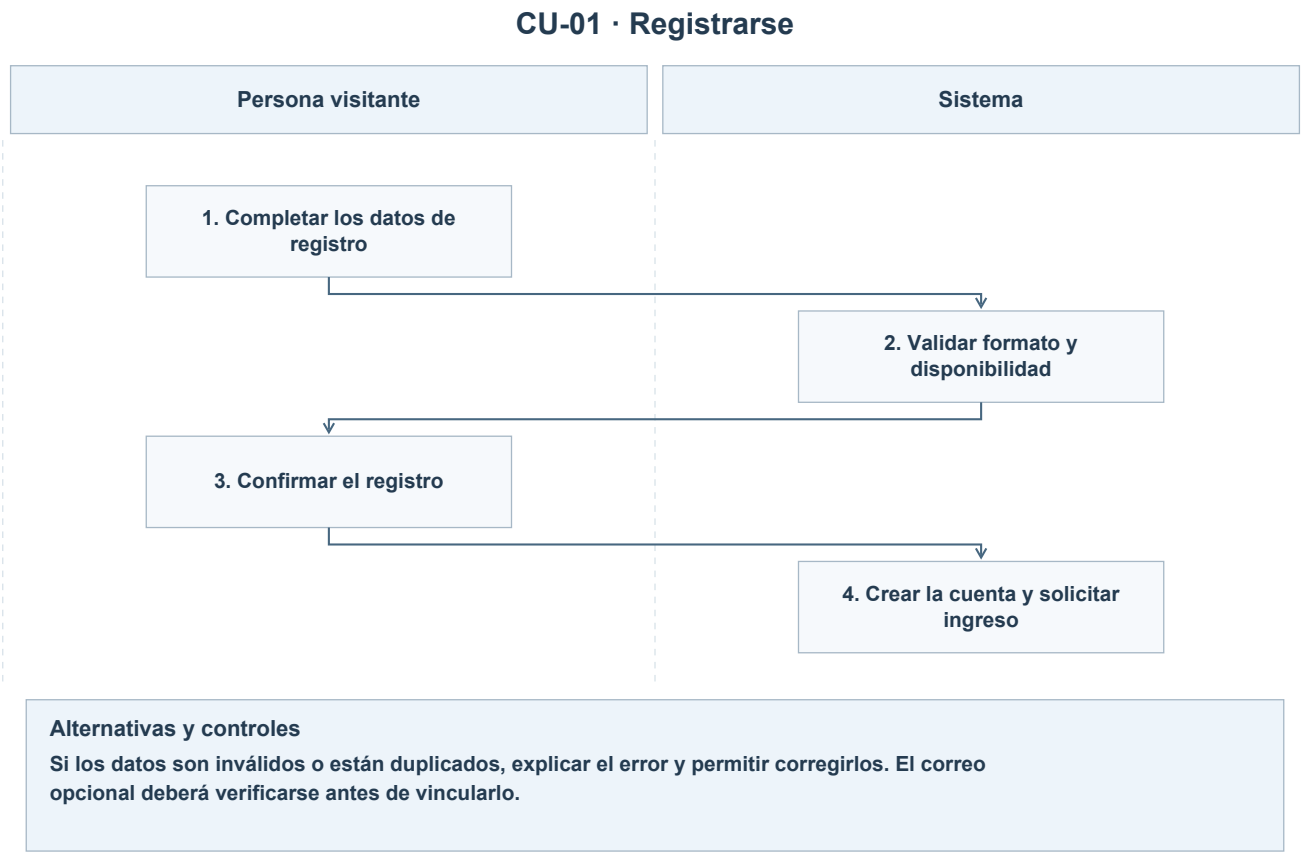
Estos recorridos concretan cómo un estudiante de la universidad puede contactar a sus compañeros, participar en grupos e intercambiar avances. Cada caso explica un objetivo y sus alternativas; los detalles internos están en P7.

## Un recorrido por caso de uso

Cada figura muestra una tarea por separado. Las columnas identifican a los participantes; los pasos numerados y las flechas indican el recorrido esperado. Las alternativas aclaran qué debe pasar si no se cumplen las condiciones.

## CU-01 · Registrarse

**Participantes:** Persona visitante, Sistema.

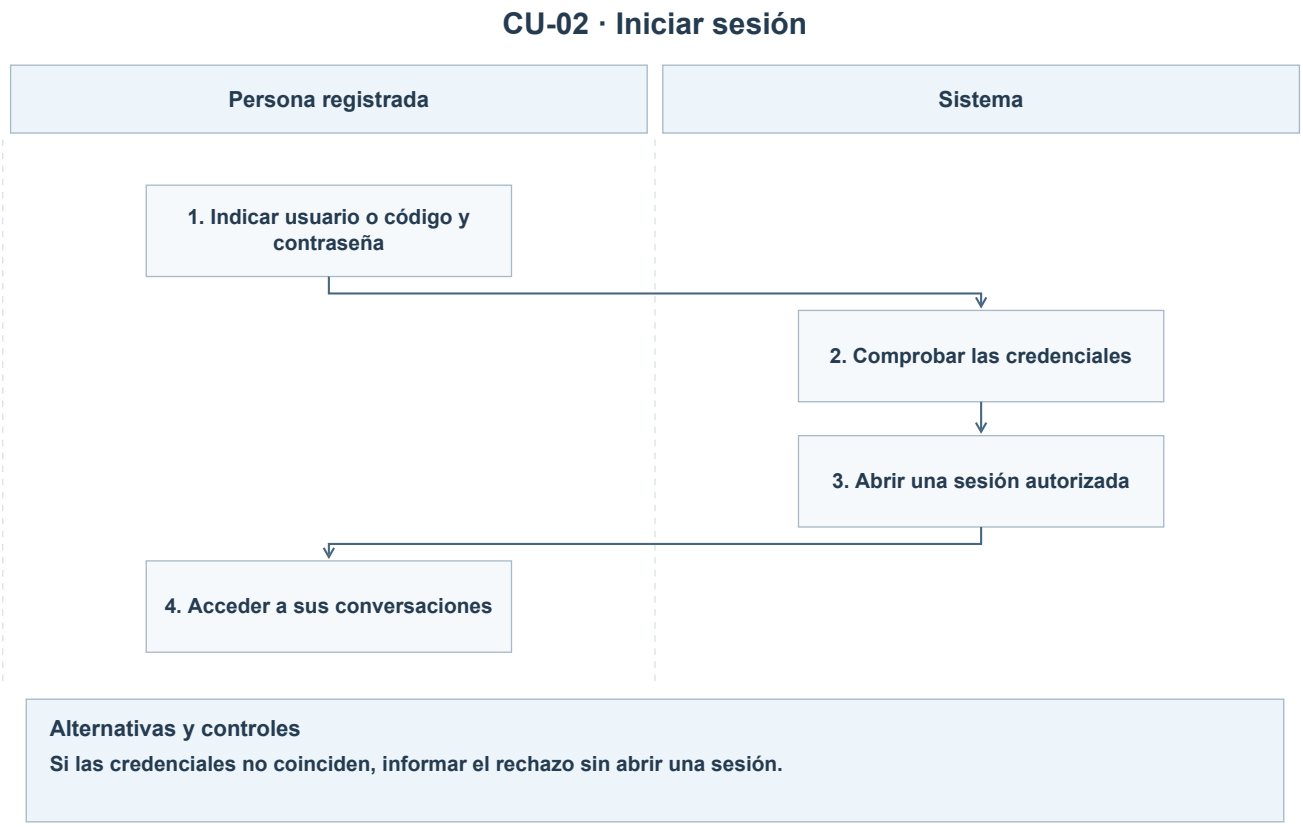



CU-01: Registrarse; flujo por participantes

Si los datos son inválidos o están duplicados, explicar el error y permitir corregirlos. El correo opcional deberá verificarse antes de vincularlo.

## CU-02 · Iniciar sesión

**Participantes:** Persona registrada, Sistema.

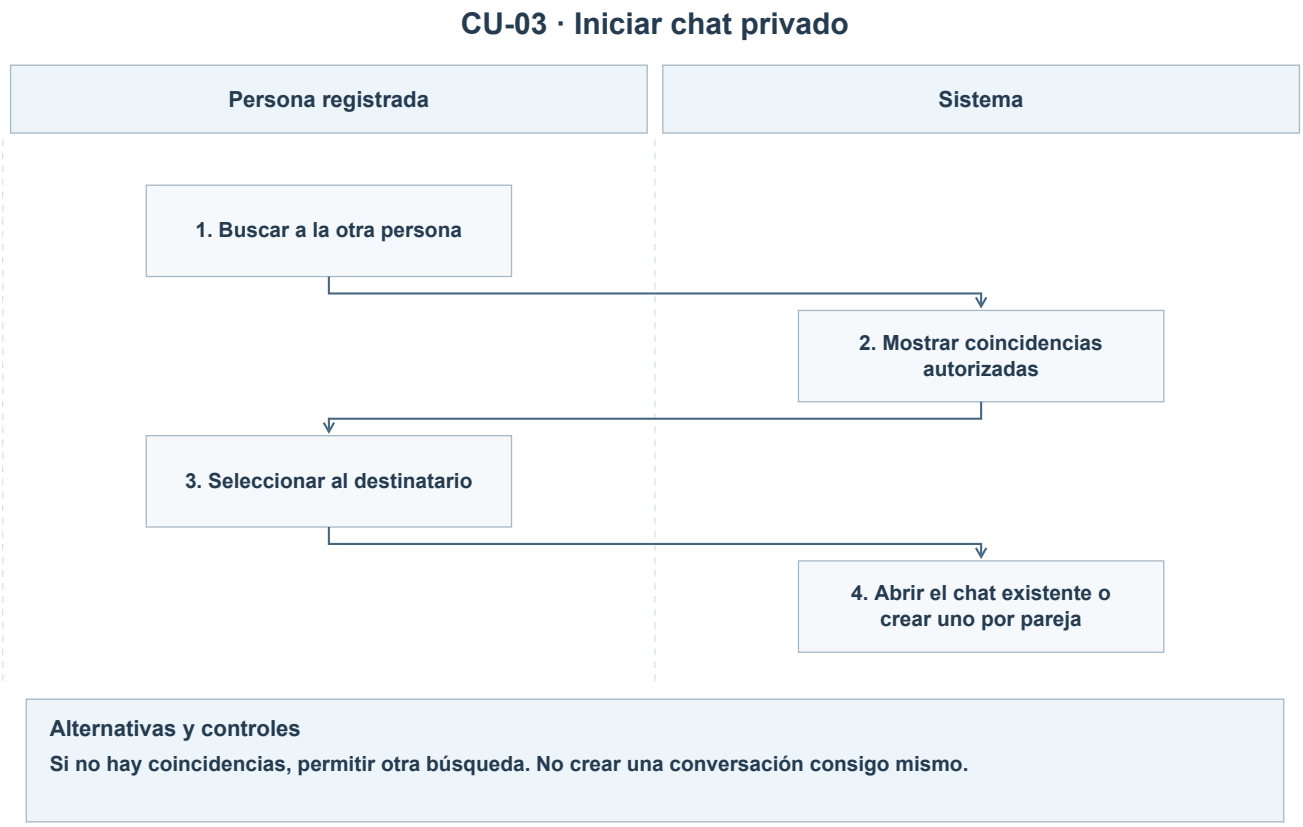



CU-02: Iniciar sesión; flujo por participantes

Si las credenciales no coinciden, informar el rechazo sin abrir una sesión.

## CU-03 · Iniciar chat privado

**Participantes:** Persona registrada, Sistema.

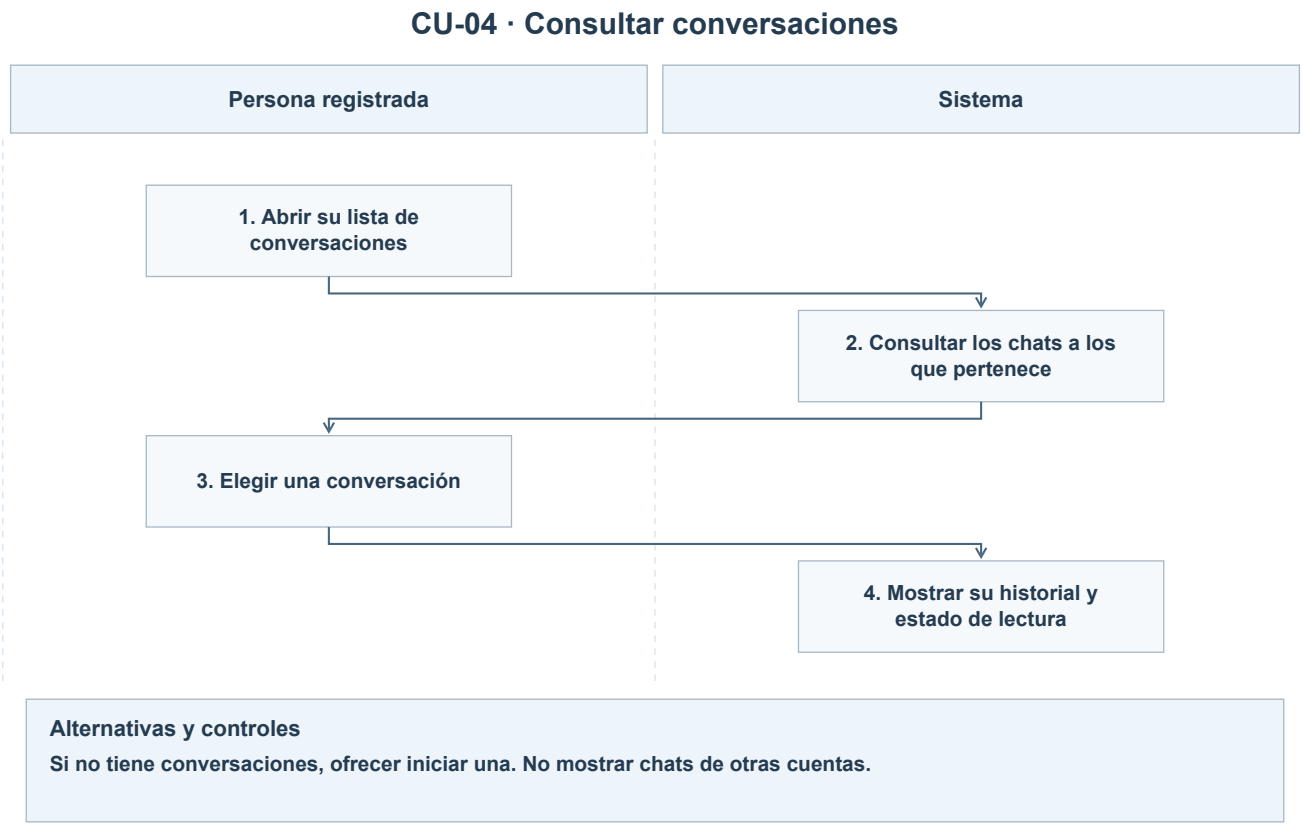



CU-03: Iniciar chat privado; flujo por participantes

Si no hay coincidencias, permitir otra búsqueda. No crear una conversación consigo mismo.

## CU-04 · Consultar conversaciones

**Participantes:** Persona registrada, Sistema.

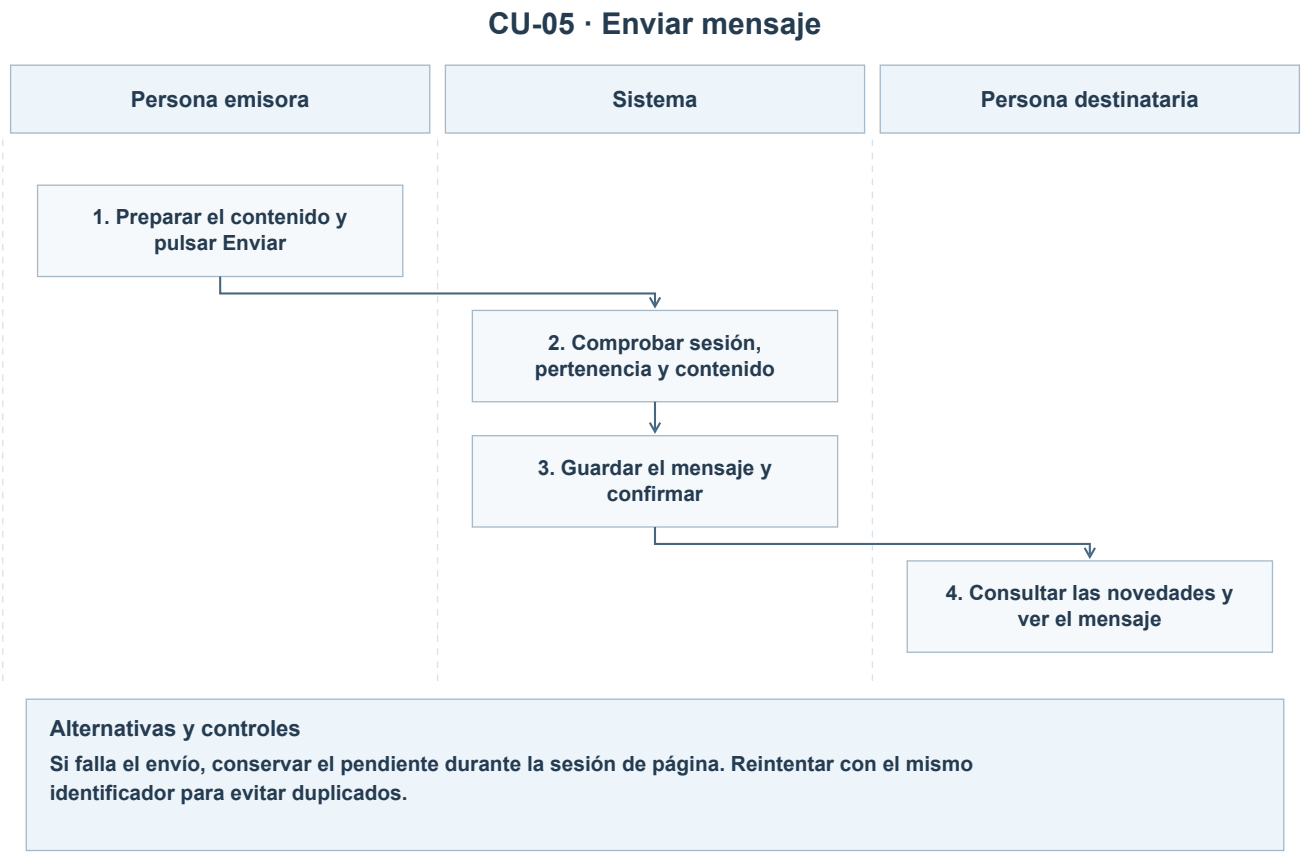



CU-04: Consultar conversaciones; flujo por participantes

Si no tiene conversaciones, ofrecer iniciar una. No mostrar chats de otras cuentas.

## CU-05 · Enviar mensaje

**Participantes:** Persona emisora, Sistema, Persona destinataria.

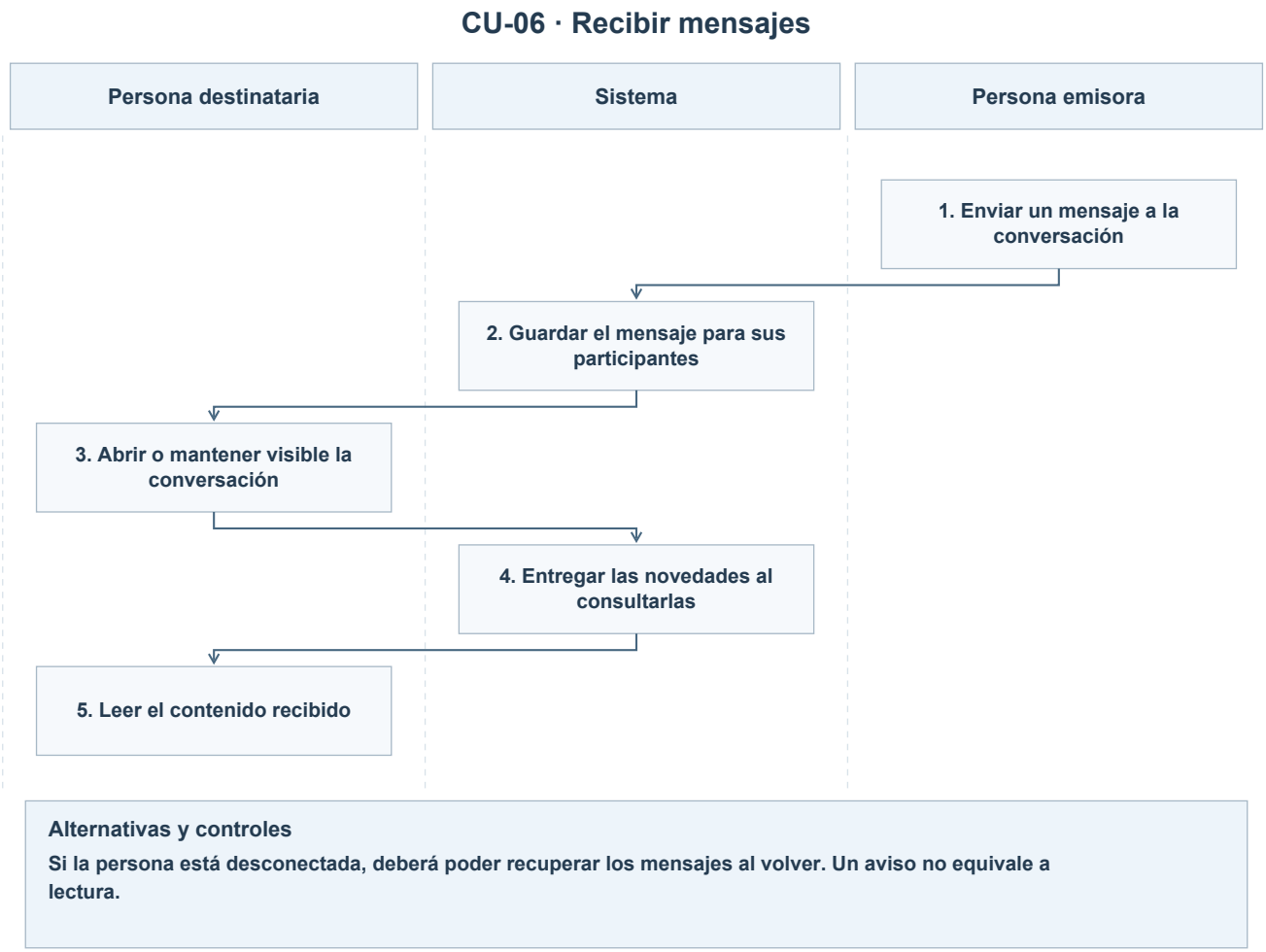



CU-05: Enviar mensaje; flujo por participantes

Si falla el envío, conservar el pendiente durante la sesión de página. Reintentar con el mismo identificador para evitar duplicados.

## CU-06 · Recibir mensajes

**Participantes:** Persona destinataria, Sistema, Persona emisora.

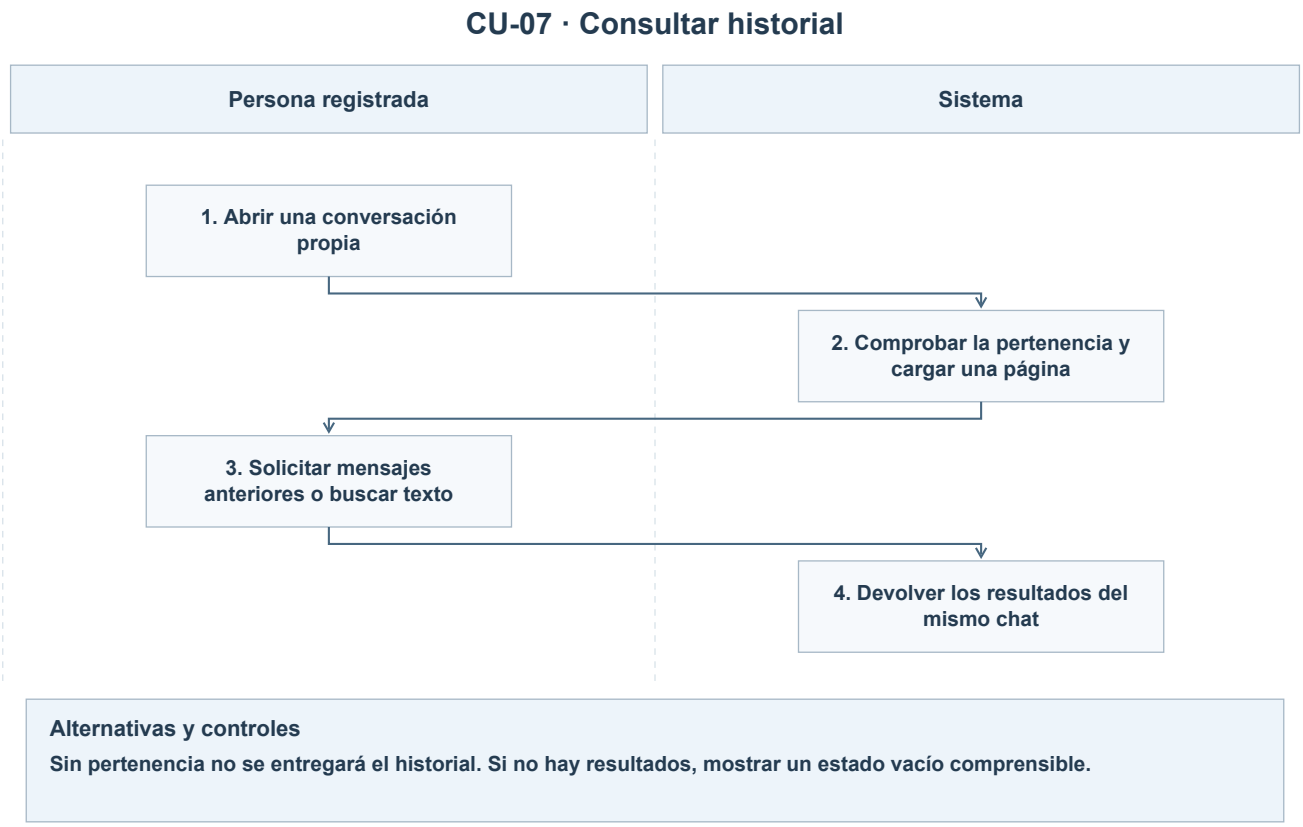



CU-06: Recibir mensajes; flujo por participantes

Si la persona está desconectada, deberá poder recuperar los mensajes al volver. Un aviso no equivale a lectura.

## CU-07 · Consultar historial

**Participantes:** Persona registrada, Sistema.

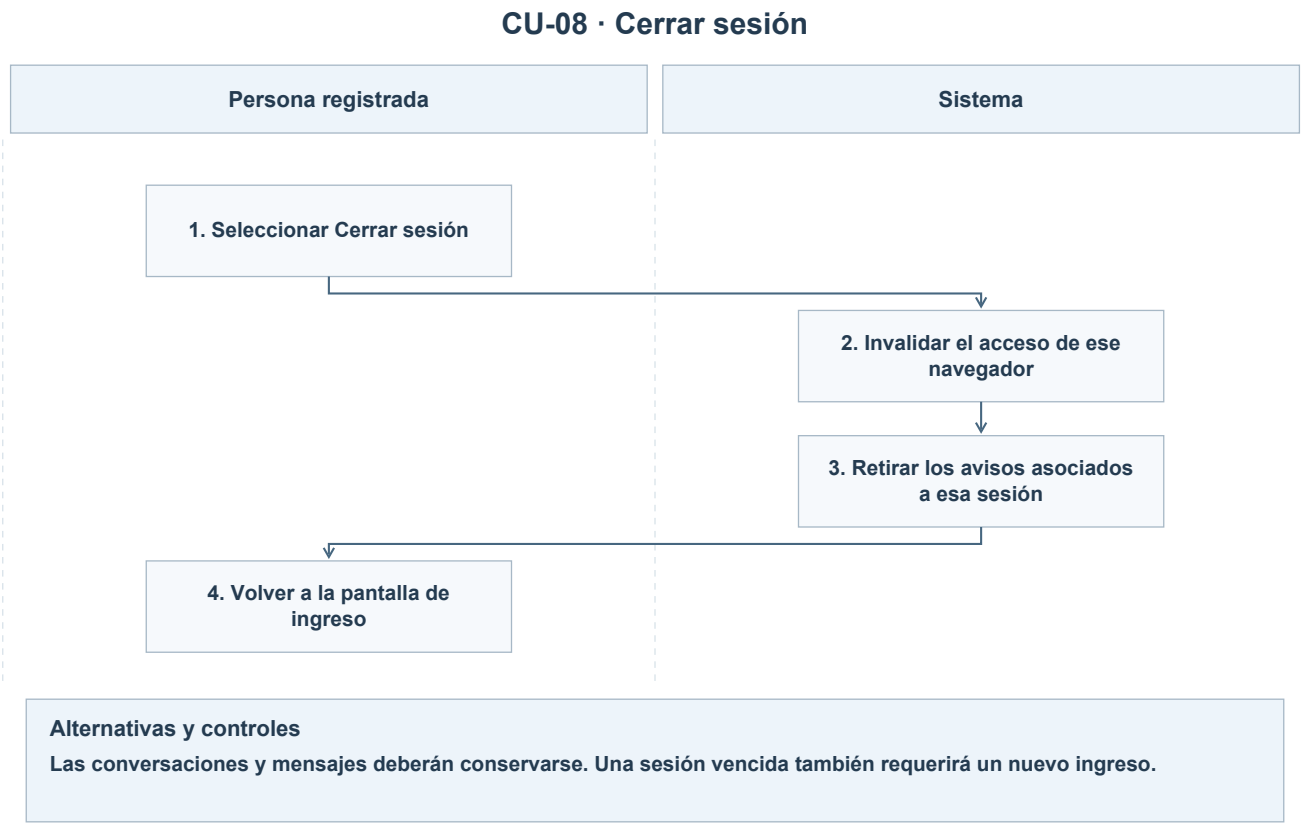



CU-07: Consultar historial; flujo por participantes

Sin pertenencia no se entregará el historial. Si no hay resultados, mostrar un estado vacío comprensible.

## CU-08 · Cerrar sesión

**Participantes:** Persona registrada, Sistema.

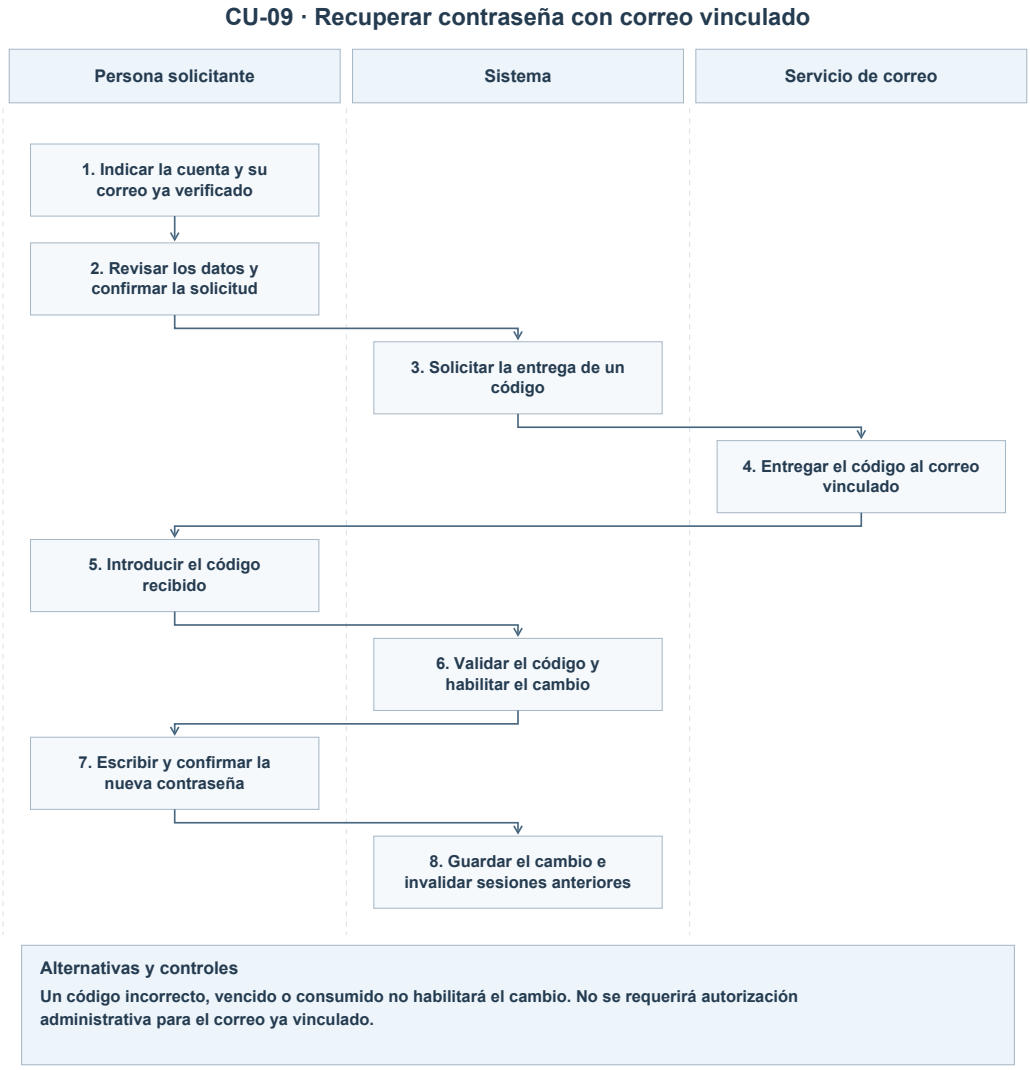



CU-08: Cerrar sesión; flujo por participantes

Las conversaciones y mensajes deberán conservarse. Una sesión vencida también requerirá un nuevo ingreso.

# CU-09 · Recuperar contraseña con correo vinculado

**Participantes:** Persona solicitante, Sistema, Servicio de correo.

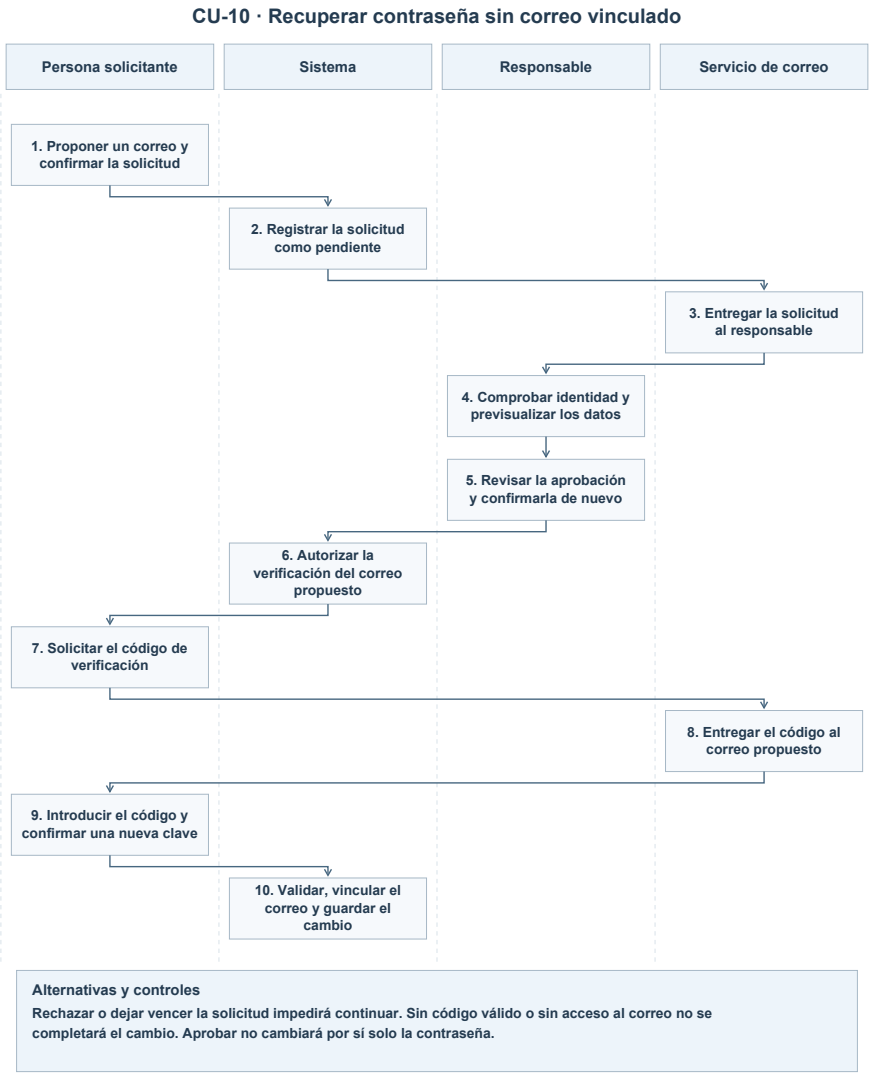



CU-09: Recuperar contraseña con correo vinculado; flujo por participantes

Un código incorrecto, vencido o consumido no habilitará el cambio. No se requerirá autorización administrativa para el correo ya vinculado.

# CU-10 · Recuperar contraseña sin correo vinculado

**Participantes:** Persona solicitante, Sistema, Responsable, Servicio de correo.

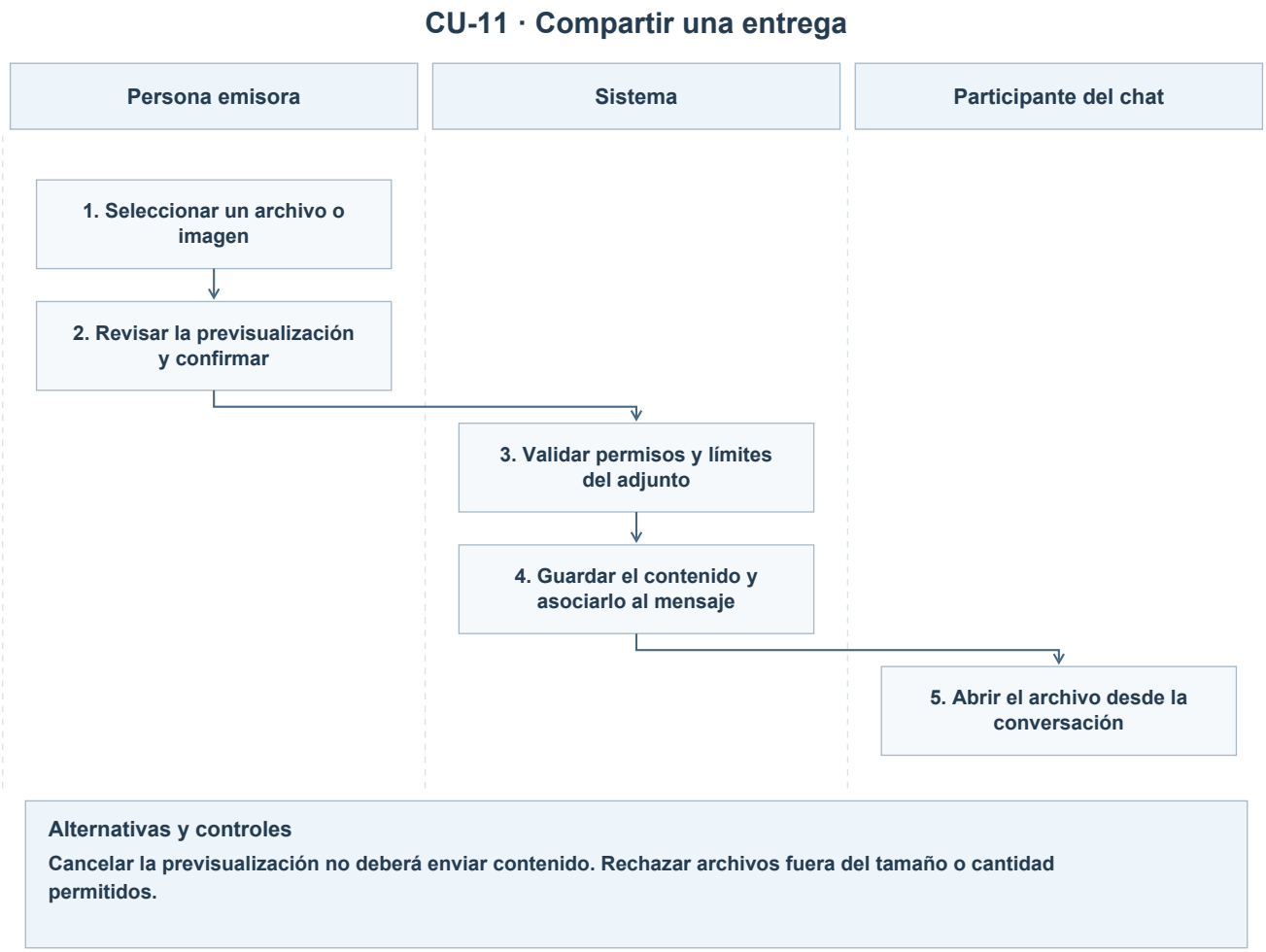



CU-10: Recuperar contraseña sin correo vinculado; flujo por participantes

Rechazar o dejar vencer la solicitud impedirá continuar. Sin código válido o sin acceso al correo no se completará el cambio. Aprobar no cambiará por sí solo la contraseña.

## CU-11 · Compartir una entrega

**Participantes:** Persona emisora, Sistema, Participante del chat.

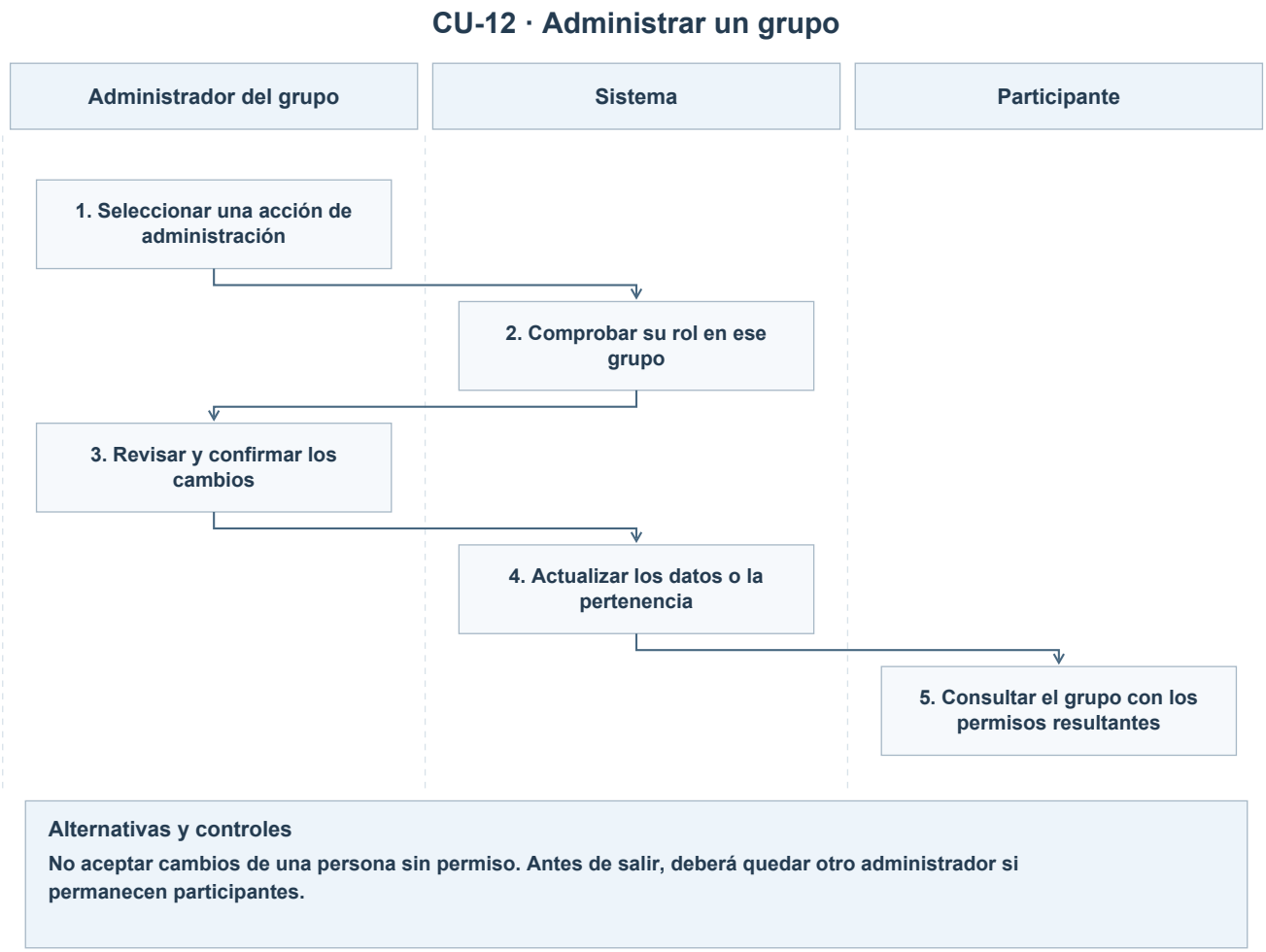



CU-11: Compartir una entrega; flujo por participantes

Cancelar la previsualización no deberá enviar contenido. Rechazar archivos fuera del tamaño o cantidad permitidos.

## CU-12 · Administrar un grupo

**Participantes:** Administrador del grupo, Sistema, Participante.

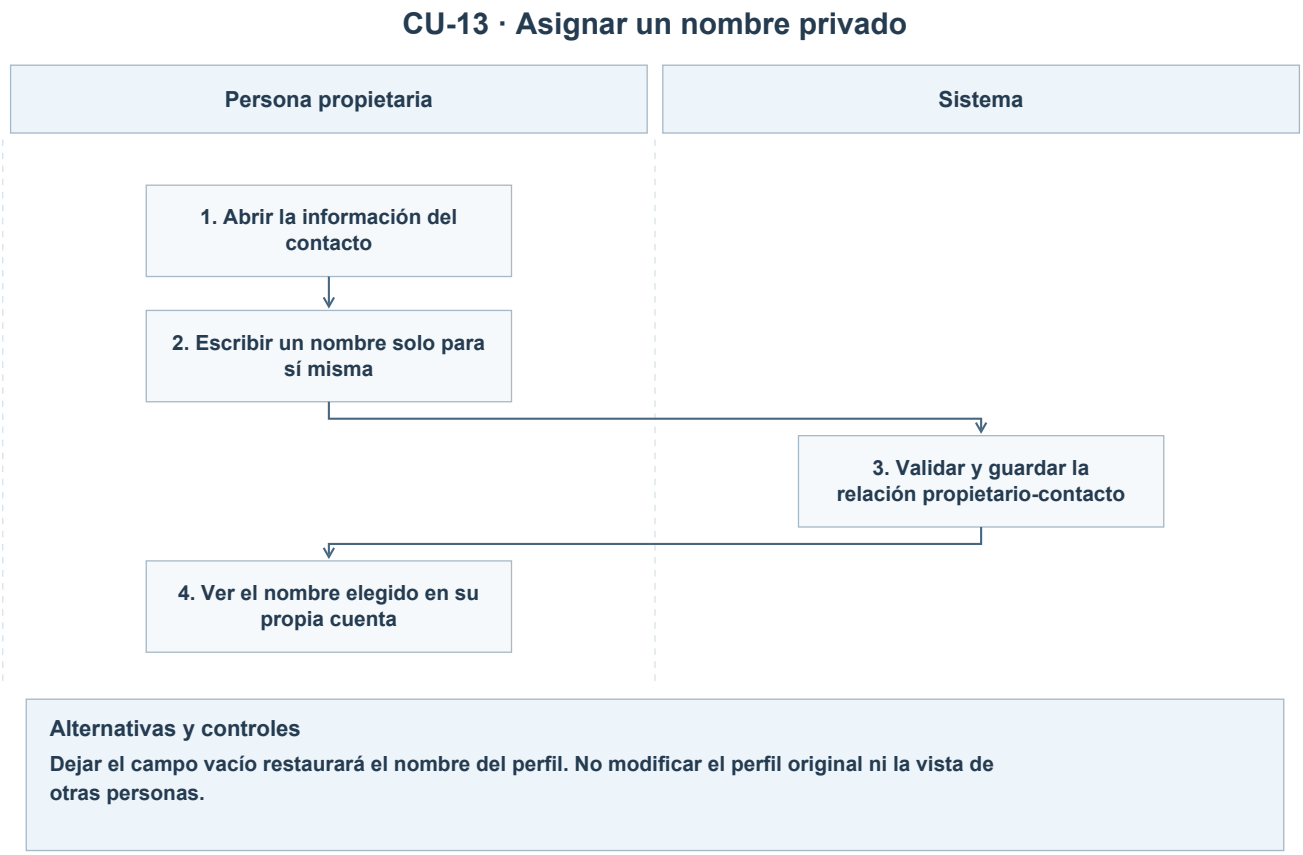



CU-12: Administrar un grupo; flujo por participantes

No aceptar cambios de una persona sin permiso. Antes de salir, deberá quedar otro administrador si permanecen participantes.

## CU-13 · Asignar un nombre privado

**Participantes:** Persona propietaria, Sistema.




CU-13: Asignar un nombre privado; flujo por participantes

Dejar el campo vacío restaurará el nombre del perfil. No modificar el perfil original ni la vista de otras personas.

# P5 · Las reglas del chat

Qué datos se aceptan y cómo se comprobará el funcionamiento.

## Datos de acceso

Estas reglas delimitan el intercambio entre estudiantes. Su validación debe realizarse también en el servidor; cambiar un campo en el navegador no concede permisos.

| Dato | Regla |
|---|---|
| Nombre completo | De 2 a 120 caracteres. |
| Código de estudiante | Exactamente 6 dígitos, almacenados como texto. Su formato no acredita por sí solo matrícula universitaria. |

| Dato | Regla |
|---|---|
| Usuario | De 3 a 20 caracteres. Empieza por una letra ASCII y admite letras, números y guion bajo. Se normaliza a minúsculas. |
| Contraseña | De 8 a 128 caracteres. La interfaz pide confirmación en registro, cambio y restablecimiento. |
| Sesión | Vencimiento absoluto de 36 horas desde el ingreso. La actividad no amplía ese plazo. |

## Conversaciones, mensajes y archivos

| Elemento | Regla |
|---|---|
| Chat privado | Dos estudiantes distintos y una sola conversación por pareja. |
| Grupo | Creación con 2 a 50 participantes. Nombre de 2 a 80 caracteres y descripción de hasta 240. Si quedan miembros, debe quedar un administrador. |
| Mensaje | Texto de hasta 1000 caracteres o al menos un adjunto. Se rechaza el contenido vacío. Solo el autor puede editar o eliminar su mensaje. |
| Adjuntos | Hasta 5 por mensaje; hasta 10 MB por archivo y 100 MB acumulados por cuenta. Las subidas descartadas y fotos reemplazadas también cuentan. |
| Foto de perfil o grupo | JPG, PNG o WebP de hasta 2 MB. |
| Historial | Páginas de 50 mensajes. La paginación limita la consulta, no la conservación del historial. |
| Perfil y nombre privado | Descripción de hasta 240 caracteres. Nombre privado de 1 a 120; dejarlo vacío elimina la personalización. |

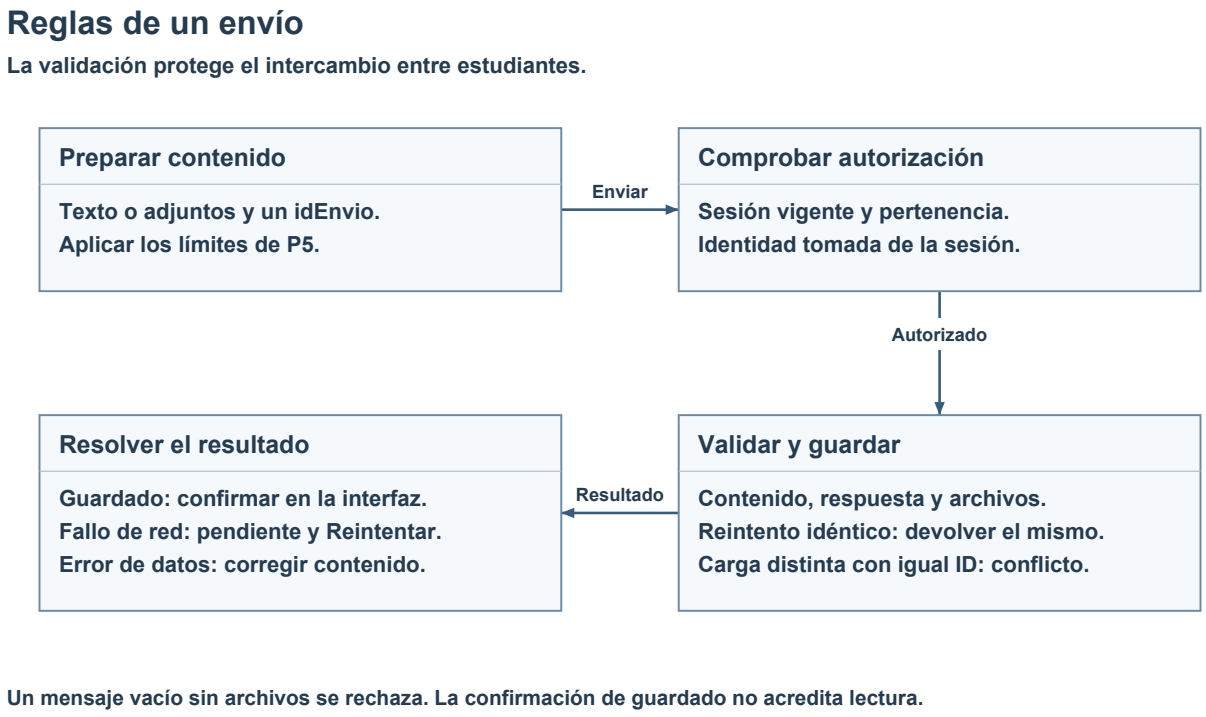
## Correo y recuperación

| Control | Regla |
|---|---|
| Código | 5 cifras, un solo uso, vigencia de 70 segundos después del envío y hasta 5 intentos. |
| Reenvíos | Esperar al menos 70 segundos; hasta 10 desafíos por hora contando correo, cuenta o navegador relacionados. |
| Solicitud excepcional | Correo nuevo sin sesión: revisión de identidad y doble confirmación administrativa. Solicitud válida durante 24 horas. |
| Cambio autorizado | Tras verificar el código, permiso para guardar la contraseña durante 10 minutos. |
| Fallo del proveedor | Mostrar el error y no confirmar un envío inexistente. La selección del proveedor debe permitir los destinatarios universitarios previstos. |

## Resultados que deben distinguirse

**Guardado** significa que la base confirmó el mensaje. **Leído** corresponde al avance de lectura del destinatario. Un aviso push solo informa que hay novedades. La secuencia que diferencia estos resultados está en P7.

## Flujo de validación




La misma validación admite texto y adjuntos. Un identificador repetido con contenido diferente produce conflicto.

## Parámetros de imágenes y voz

| Elemento | Regla de la interfaz |
|---|---|
| Nota de voz | Grabación de hasta 5 minutos, con permiso de micrófono y contexto HTTPS o localhost. También se puede adjuntar un archivo de audio. |
| Foto del perfil | Encuadre 1:1 y salida PNG de 512 por 512 píxeles antes de guardar. |
| Imagen editada | Recorte original, 1:1, 4:3, 3:4 o 16:9; salida de hasta 2048 píxeles en el lado mayor. |
| Reproducción | Velocidades 1, 1,5 y 2; pausa, avance y descarga dentro del chat. |

# P6 · Los elementos del sistema

Las cuentas, conversaciones y mensajes que forman el chat.

## Conceptos del dominio

El estudiante es la persona que utiliza AzChat; `usuarios` será el nombre técnico de sus cuentas. Este modelo describe lo necesario para que los estudiantes se comuniquen antes de decidir las columnas SQL.

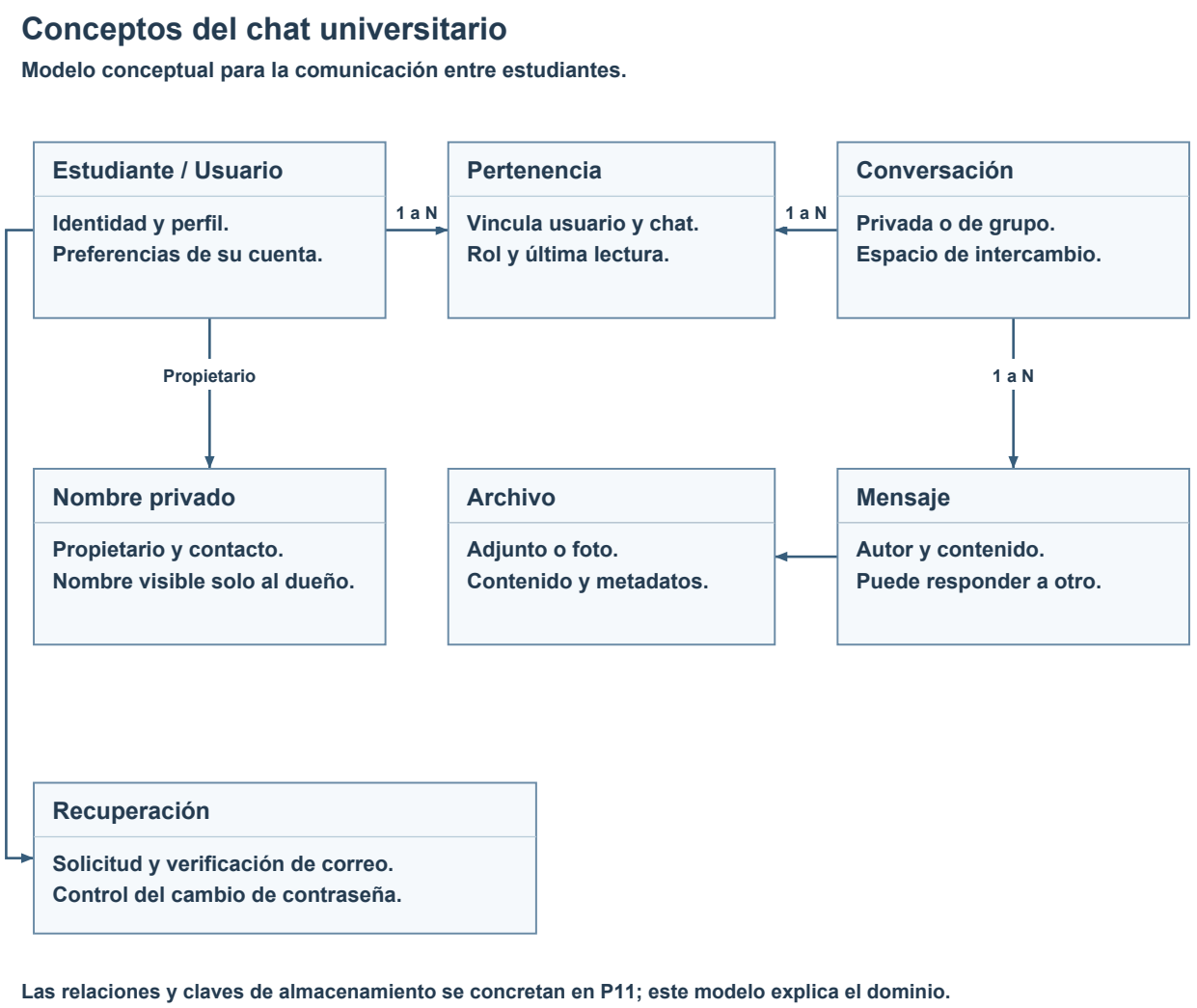
| Concepto | Qué representa |
|---|---|
| Estudiante o usuario | Identidad con perfil y preferencias propias. |
| Conversación | Espacio de intercambio privado o grupal. |
| Pertenencia | Relación entre estudiante y conversación, con rol y lectura propios. |
| Mensaje | Intervención de un autor; puede responder a otra e incluir archivos. |
| Archivo | Contenido adjunto o foto con metadatos. |
| Nombre privado | Nombre que un estudiante asigna a un contacto solo para su propia vista. |
| Recuperación | Solicitud de autorización y desafío de verificación del correo. |

## Separaciones que importan

Un estudiante conserva su identidad al cerrar sesión. Un grupo puede continuar cuando sale uno de sus miembros. Fijar, silenciar o leer es una decisión de cada participante, no un cambio común del grupo. Por eso la pertenencia tiene información propia.

Un nombre privado no modifica el perfil del compañero. Una solicitud aprobada tampoco demuestra acceso al correo. Separar estos conceptos evita mezclar preferencias, permisos y pruebas de identidad. Las relaciones persistentes están en P11.

## Modelo conceptual del sistema




El modelo incluye conversaciones privadas y grupos; las tablas y sus claves se especifican en P11.

# P7 · El recorrido de un mensaje

Qué ocurre desde que se escribe un mensaje hasta que se recibe.

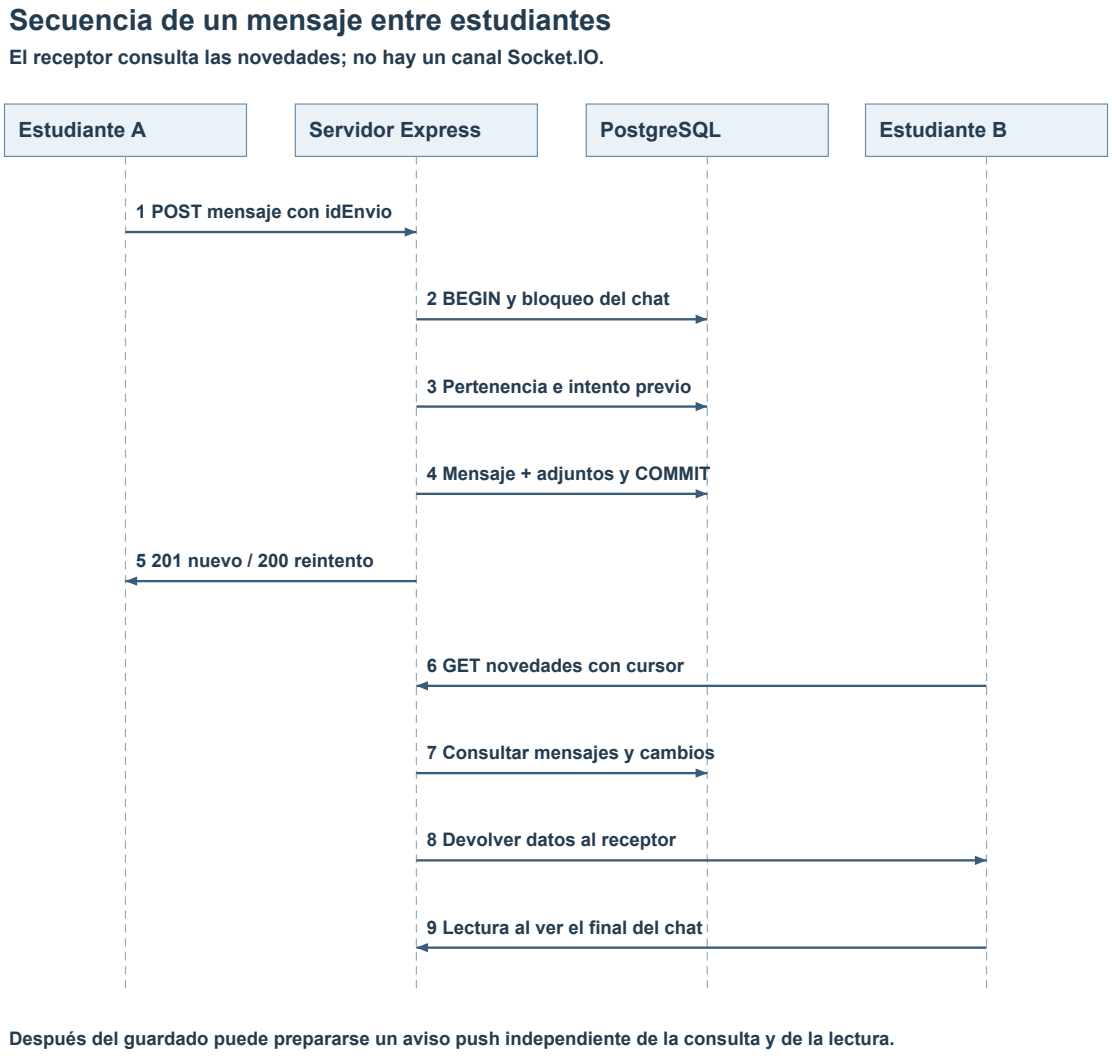
## Del estudiante emisor al historial

Cuando un estudiante envía un avance a otro compañero o a su grupo, el sistema debe conservarlo antes de confirmarlo y entregarlo únicamente a sus participantes.

1. **Preparar:** la interfaz conserva el borrador y genera un UUID idEnvio. Si hay adjuntos, los sube primero y recoge sus identificadores.
2. **Solicitar:** envía texto, identificador, respuesta opcional y lista de archivos a POST /api/conversaciones/:id/mensajes. El emisor se obtiene de la sesión.
3. **Autorizar:** el servidor comprueba sesión, pertenencia y formato. Dentro de una transacción bloquea la conversación y vuelve a comprobar pertenencia.

4. **Reconocer un reintento:** busca la pareja emisor e identificador. Si chat, texto, respuesta y archivos coinciden, devuelve el mensaje existente; si cambian, responde con conflicto.
5. **Guardar:** comprueba que la respuesta pertenece al mismo chat y que los archivos son del emisor, de esa conversación y no están vinculados. Inserta mensaje y asociaciones; confirma la transacción.
6. **Confirmar:** devuelve el mensaje con su ID y fecha de servidor. Un envío nuevo responde 201; un reintento reconocido, 200. La interfaz sustituye el pendiente por el registro confirmado.
7. **Recuperar:** el navegador del destinatario consulta las novedades. El servidor filtra por pertenencia y la interfaz incorpora o actualiza cada mensaje por su ID.

## Secuencia interna




La base confirma la transacción antes de la respuesta. El destinatario recibe contenido al consultar la API; el aviso push es independiente.

## Sincronización, historial y lectura

El cliente espera aproximadamente 1,8 segundos después de terminar cada ciclo antes del siguiente, o 6 segundos si la pestaña está oculta. Consulta la lista de conversaciones y los mensajes del chat abierto. No mantiene un canal WebSocket o Socket.IO.

La consulta inicial trae la última página. antes solicita mensajes anteriores, despues trae los posteriores al cursor y cambios recupera ediciones, eliminaciones y respuestas afectadas desde una fecha del servidor. La interfaz concilia por ID para no dibujar copias repetidas.

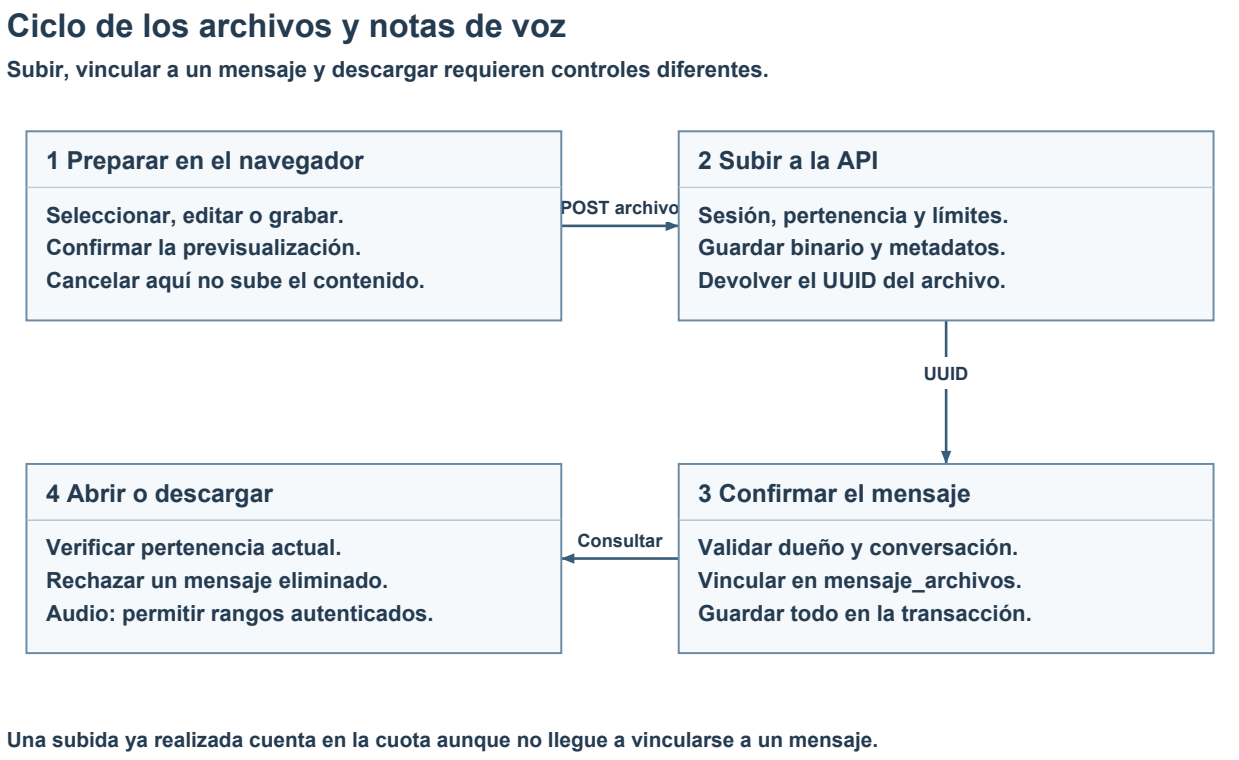
La lectura avanza con POST /api/conversaciones/:id/lectura al consultar con el chat visible y cerca del final del historial, o al llegar al final mediante desplazamiento. La base usa GREATEST para que el indicador no retroceda. La vista móvil oculta no marca lectura durante la consulta.

## Desconexión y reintentos

Si se pierde la respuesta, el estudiante puede ver un fallo aunque el mensaje ya esté guardado. Reintentar reutiliza exactamente el intento original. La recuperación de Internet reactiva la consulta del historial; no reenvía automáticamente borradores.

Borradores y pendientes viven en la memoria de la página. Cambiar de chat los conserva durante esa apertura; recargar o cerrar puede perderlos. Si la transacción falla, se revierte el mensaje junto con sus asociaciones; los archivos que ya se subieron permanecen almacenados.

## Archivos, imágenes y audio




El contenido binario se sube antes del mensaje. La descarga comprueba permisos en cada petición, incluido el avance por rangos de un audio.

Las fotos de perfil pueden consultarse con sesión. Los adjuntos requieren pertenencia actual al chat; antes de vincularse, solo su propietario puede abrirlos. Un mensaje eliminado impide descargarlos. La nota de voz sigue el mismo circuito de adjunto, con captura mediante permiso de micrófono.

### Avisos y presencia

Después de confirmar un mensaje nuevo se preparan avisos push para miembros con suscripción y sesión vigentes que no hayan silenciado el chat. Un fallo de ese aviso no revierte el mensaje. El service worker avisa a ventanas abiertas y puede mostrar la notificación si ninguna está enfocada.

La presencia se actualiza por sesión mediante latidos mientras la página está visible. Al ocultarla o salir se envía una señal de pausa. Es una estimación de actividad, no una prueba de lectura ni de disponibilidad física del estudiante.

# P8 · Cómo se reparte el trabajo

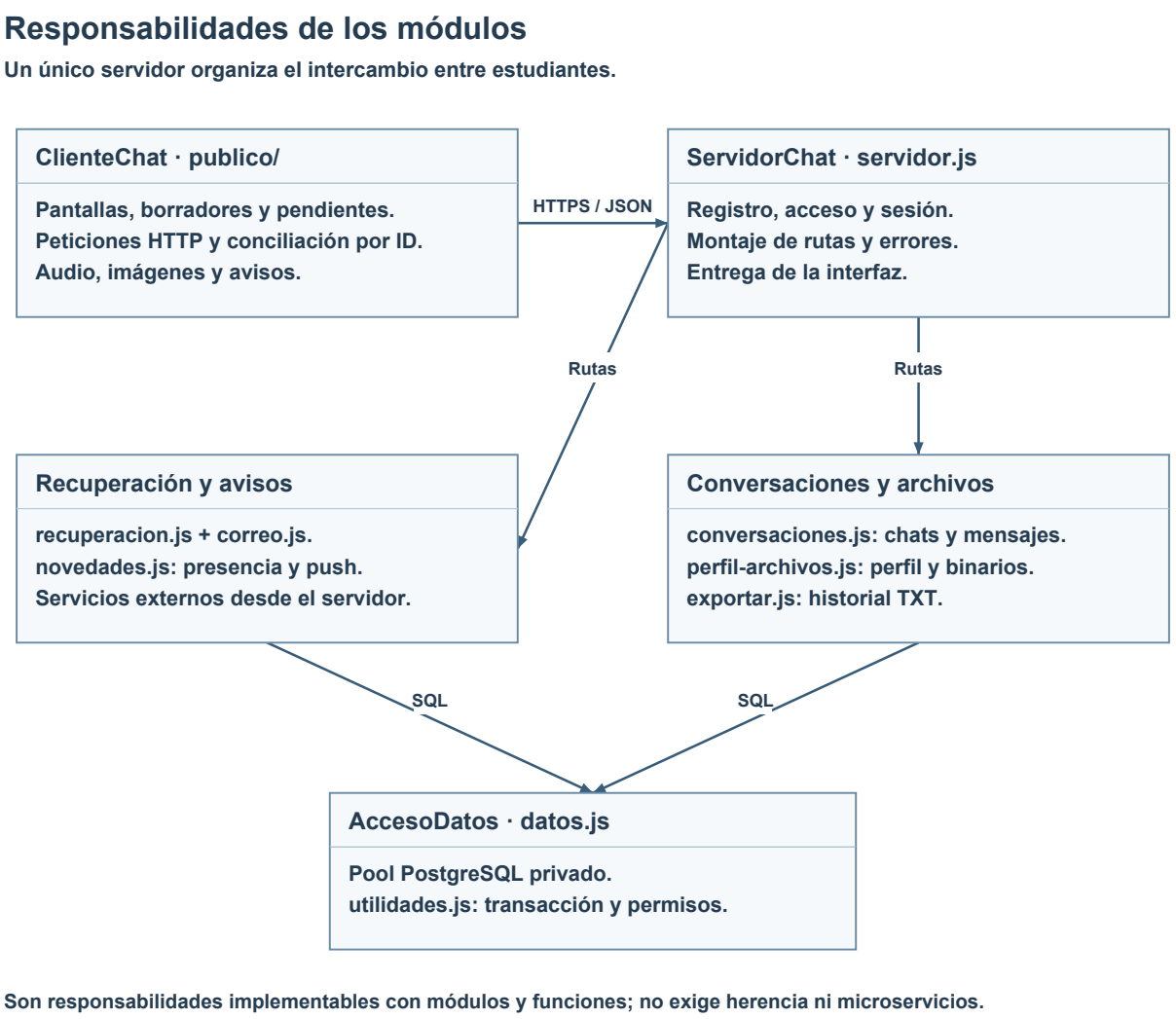
Responsabilidades propuestas para las partes del sistema.

## Módulos y responsabilidades

El diseño divide el trabajo necesario para intercomunicar a los estudiantes sin separar cada función en un servidor. Los nombres siguientes concretan la organización de módulos JavaScript y funciones, aunque la representación use responsabilidades de clases.

| Módulo previsto | Responsabilidad |
|---|---|
| publico/app.js y complementos | Acceso, navegación, estado de conversaciones, borradores, imágenes, audio e indicaciones. |
| servidor.js | Crear Express, servir la web, registrar cuentas, verificar credenciales y administrar sesiones. |
| conversaciones.js | Chats privados y grupos, miembros, roles, mensajes, lectura e historial. |
| perfil-archivos.js | Perfil, ajustes, nombres privados, cambio de contraseña y binarios autenticados. |
| recuperacion.js y correo.js | Desafíos de correo, solicitudes, decisión administrativa y transporte de correo. |
| novedades.js y publico/sw.js | Presencia, suscripción Web Push, entrega y apertura de avisos. |
| exportar.js | Exportación TXT con verificación de pertenencia. |
| datos.js y utilidades.js | Pool SQL, permisos reutilizables, transacciones y manejo de errores. |

## Colaboración entre responsabilidades




El cliente solicita una acción; los módulos del servidor autorizan y consultan PostgreSQL. Los archivos del navegador no reciben credenciales privadas.

## Contrato de la API

Las rutas usan el prefijo /api. La tabla agrupa las operaciones del diseño; :id es una conversación, :mid un mensaje y :uid un usuario.

| Ruta u operación | Datos y resultado |
|---|---|
| POST /registro; POST, GET y DELETE /sesion | Crear cuenta, ingresar, comprobar acceso y salir. |
| GET /usuarios?q=… | Buscar por nombre o usuario con sesión. |
| GET y POST /conversaciones | Listar chats propios y abrir una pareja o crear grupo. |
| PATCH /conversaciones/:id | Modificar datos del grupo con rol de administrador o fijado y silenciado propios. |

| Ruta u operación | Datos y resultado |
|---|---|
| GET, POST /conversaciones/:id/miembros; PATCH, DELETE …/miembros/:uid | Consultar integrantes, añadirlos, cambiar roles o retirar una pertenencia según permisos. |
| GET y POST /conversaciones/:id/mensajes | Consultar con cursores o guardar el intento validado. |
| PATCH y DELETE …/mensajes/:mid; POST …/lectura | Editar o eliminar un mensaje propio y avanzar la lectura. |
| PATCH /perfil; PATCH /perfil/contrasena | Gestionar cuenta propia y revocar otras sesiones al cambiar la clave. |
| PUT /contactos/:id/nombre | Guardar o quitar un nombre privado para un contacto en la cuenta propia. |
| POST /archivos; GET /archivos/:id | Subir contenido binario o descargarlo tras la autorización. |
| GET /exportar/chats.txt | Exportar todos los chats propios o una selección autorizada. |
| POST /registro/correo/*, /correo/* y /recuperacion/* | Emitir y verificar códigos; solicitar y completar recuperación. |
| POST /presencia; GET /notificaciones/clave; POST, DELETE /notificaciones/suscripcion | Actualizar actividad y gestionar avisos por dispositivo. |

### Errores y frontera entre capas

Una petición inválida produce un error comprensible sin guardar contenido parcial. La API diferencia datos inválidos (400), sesión ausente (401), permiso insuficiente (403), recurso no disponible (404), conflicto (409), vencimiento (410), límite de tamaño (413), exceso de solicitudes (429) e indisponibilidad del correo (503).

Ocultar un botón no es autorización. Los controles de sesión, autoría, propiedad de archivos y pertenencia deben repetirse en el servidor. Las consultas SQL usan parámetros; los detalles internos de un fallo no se entregan al navegador.

# P9 · Cómo se conectan sus partes

La relación entre el navegador, el servidor y la base de datos.

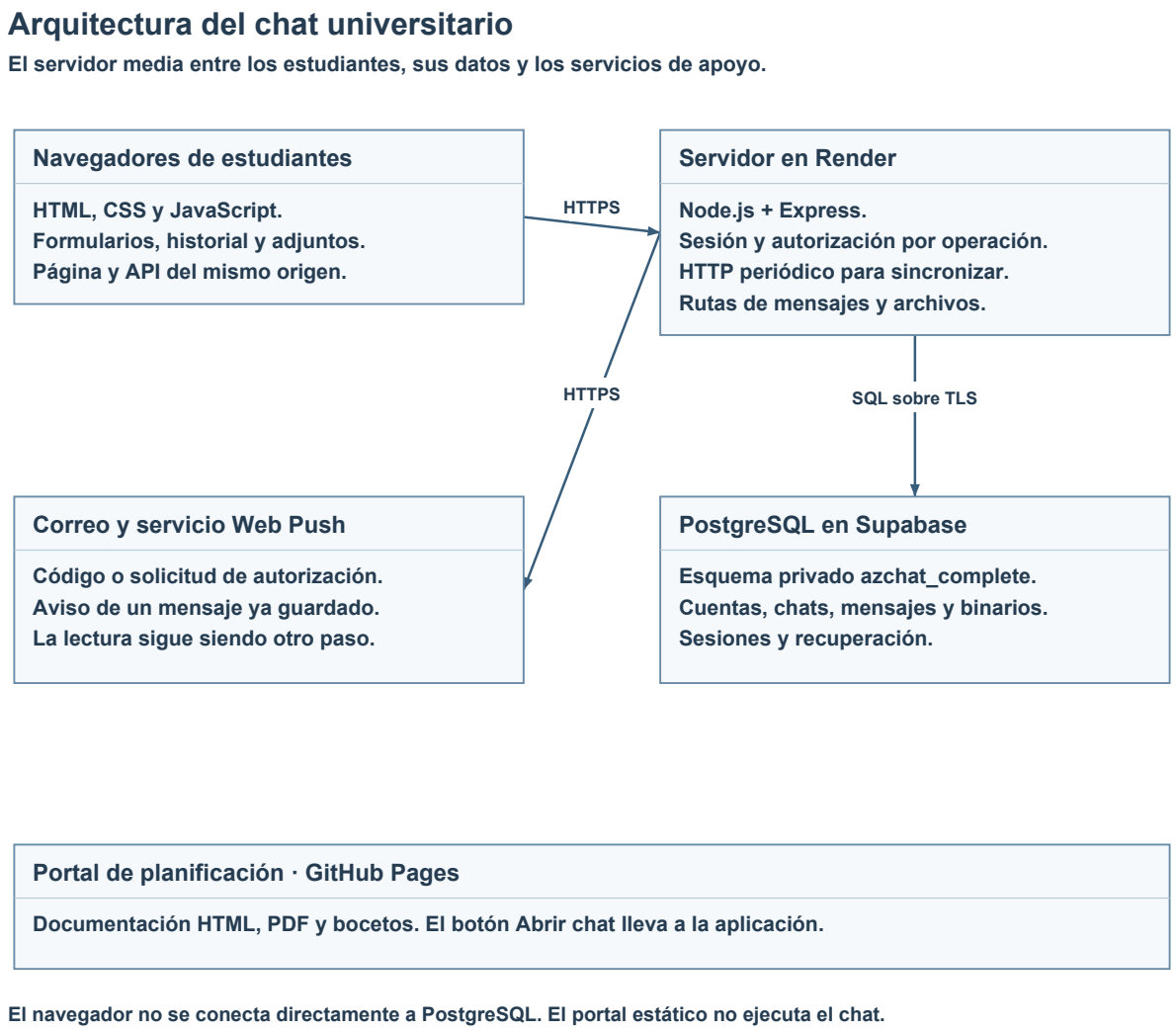
### Implementación de la arquitectura

En Complete 2.9 la presentación utiliza HTML, CSS y JavaScript; Node.js con Express atiende las rutas HTTP; el módulo pg conecta el servidor con PostgreSQL alojado en Supabase. La autenticación y las sesiones las administra AzChat, no el servicio Supabase Auth. Las novedades se consultan periódicamente con Fetch, sin Socket.IO en Complete.

Render aloja el servidor y GitHub privado conserva su código. Google Cloud configura el acceso OAuth 2.0 y Gmail API envía desde la cuenta autorizada del responsable. Esta API recibe la petición del servidor; las credenciales de correo no se envían al navegador. El paso 12 explica el proceso completo.

## Arquitectura cliente servidor

Los estudiantes abren la aplicación desde sus navegadores. Una única aplicación Node.js con Express entrega los archivos de interfaz y atiende la API. PostgreSQL administrado en Supabase conserva los datos; el servidor se ubica en Render. GitHub Pages entrega únicamente la documentación y los bocetos.




La comunicación del chat pasa por Express. La consulta periódica del historial se explica en P7.

## Capas y límites de confianza

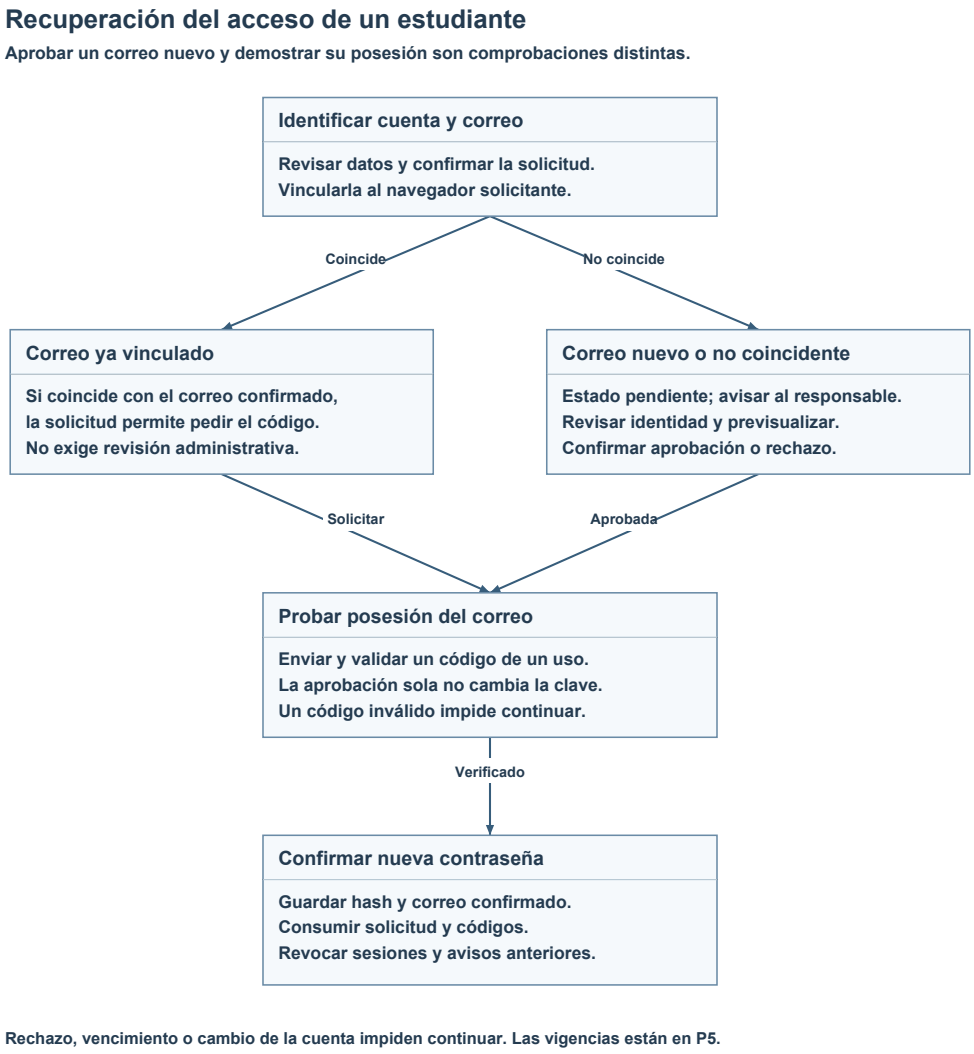
| Capa | Decisión |
| --- | --- |
| Presentación | HTML, CSS y JavaScript sin un framework de componentes obligatorio. Web y API comparten origen. |
| Aplicación | Reglas y permisos centralizados en Express. Sesiones con express-session y connect-pg-simple; contraseñas con Argon2id. |
| Persistencia | Acceso mediante pg y pool pequeño. Esquema azchat_complete independiente; consultas parametrizadas y TLS con validación de certificado. |
| Servicios de apoyo | Proveedor de correo por HTTPS y Web Push invocados desde el servidor. La autorización sigue siendo responsabilidad de AzChat. |

## Protección del acceso

La cookie contiene un identificador firmado; los datos de sesión están en PostgreSQL. Se regenera al ingresar y usa HttpOnly, SameSite=Lax y Secure en producción. El servidor comprueba el vencimiento absoluto definido en P5 y el origen de las operaciones que modifican datos.

Las credenciales de PostgreSQL, el secreto de sesión y las claves de correo y push pertenecen a configuración privada del servidor. El esquema no debe exponerse a roles públicos de Supabase; RLS complementa la separación. La autorización por estudiante y conversación se aplica en Express. El diseño usa PostgreSQL de Supabase, sin depender de Supabase Auth, Realtime ni un bucket público para archivos.

## Recuperación de una cuenta estudiantil




Ambos caminos exigen acceso al correo. Aprobar la solicitud es distinto de verificar el código y de guardar la nueva contraseña.

El desafío se vincula al propósito, al navegador y, cuando corresponde, a la cuenta y a su solicitud. Se guarda un HMAC del código, no el código en claro. El servidor bloquea los registros durante la verificación y conserva los intentos incorrectos.

Para un correo no coincidente, el responsable debe contrastar la identidad por un medio conocido antes de decidir. El enlace abre una previsualización y luego exige confirmación. Al completar el restablecimiento se consume la solicitud, se invalidan las demás y se revocan sesiones y suscripciones anteriores.

## Alojamiento y operación

El despliegue necesita las dependencias fijadas, el esquema preparado y la configuración privada. El servidor escucha el puerto del entorno y expone /salud, que verifica una consulta simple a la base. Este endpoint comprueba disponibilidad básica, no todas las funciones del chat.

Un alojamiento con suspensión por inactividad puede requerir espera al arrancar. La evaluación debe separar esa espera de la respuesta con servidor activo. Instalar la web añade un acceso en el dispositivo y permite avisos compatibles; el service worker no mantiene una copia del historial ni una cola de envío sin

conexión.

# P10 · Cómo se usa AzChat

Bocetos, navegación y confirmaciones de las pantallas.

## Organización de las pantallas

En computadora se mantiene una barra lateral reducida que se amplía al pasar el puntero. Al abrir Ajustes aparece un panel integrado entre esa barra y la conversación; no cubre el historial con una ventana emergente. La conversación utiliza el espacio restante.

En celular se muestra una sola vista principal. Chats, Perfil y Ajustes se eligen desde la barra inferior; al abrir una conversación, el panel se desplaza hasta el chat y el listado queda oculto. Volver regresa al listado. El contenido largo se desplaza verticalmente dentro de su área.

## Ayuda breve y confirmaciones

El botón **!** permitirá consultar indicaciones sin llenar la pantalla de instrucciones. Los errores y las consecuencias de acciones importantes deberán permanecer visibles. El correo de recuperación será opcional y tendrá su propia verificación.

## Bocetos del diseño

Estos bocetos de baja y media fidelidad conservan la organización del sistema actual sin reproducir sus detalles finales. Se usan contornos simples, avatares de muestra y bloques de contenido: el propósito es explicar jerarquía y navegación, no presentar capturas de producción.

Son una revisión del diseño dentro del proceso iterativo. Permiten reconocer la continuidad entre las ideas de distribución y la aplicación, pero no se atribuyen retrospectivamente todos los detalles a la planificación inicial.

# Esquemas de las pantallas

## 01 · Distribución de escritorio

AzChat · Diseño revisado · Elementos simplificados y datos ficticios




Barra, panel de ajustes opcional y conversación comparten el ancho.

## 02 · Un orden común para los dos temas

AzChat · Diseño revisado · Elementos simplificados y datos ficticios

Tema claro




Tema oscuro

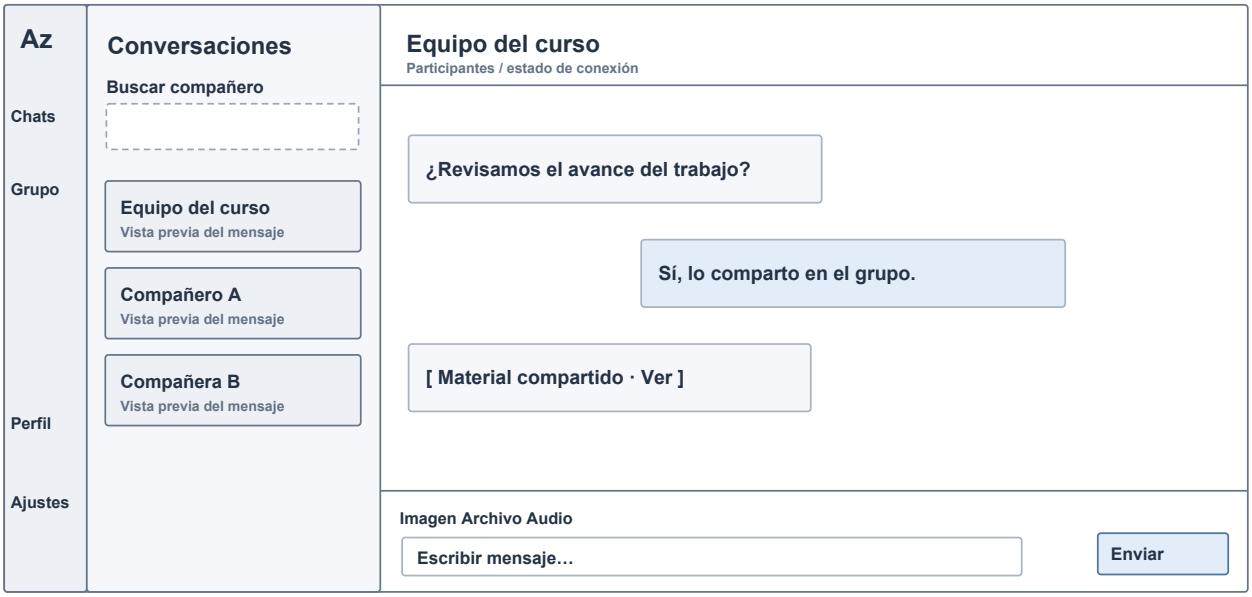



**Se conserva la jerarquía. Los tonos solo orientan; no fijan el acabado final.**

El mismo orden se conserva con superficies claras u oscuras.

## 03 · Chats privados y grupos

AzChat · Diseño revisado · Elementos simplificados y datos ficticios

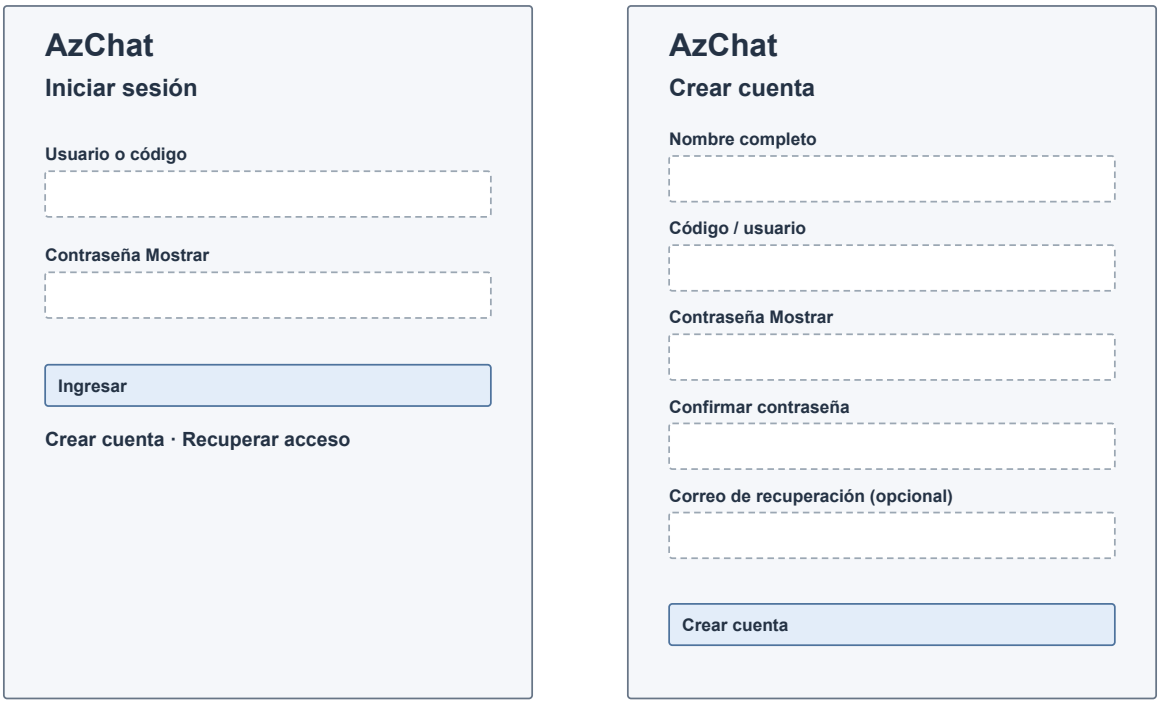



**La barra puede ampliar su contenido; el grupo reúne mensajes de sus participantes.**

Lista, grupo y mensajes de ejemplo, con herramientas separadas.

## 04 · Acceso y creación de cuenta

AzChat · Diseño revisado · Elementos simplificados y datos ficticios




**Panel reducido sobre un fondo ambiental. No se representan efectos ni animaciones finales.**

Panel reducido, confirmación de contraseña y correo opcional.

## 05 · Una vista móvil a la vez

AzChat · Diseño revisado · Elementos simplificados y datos ficticios



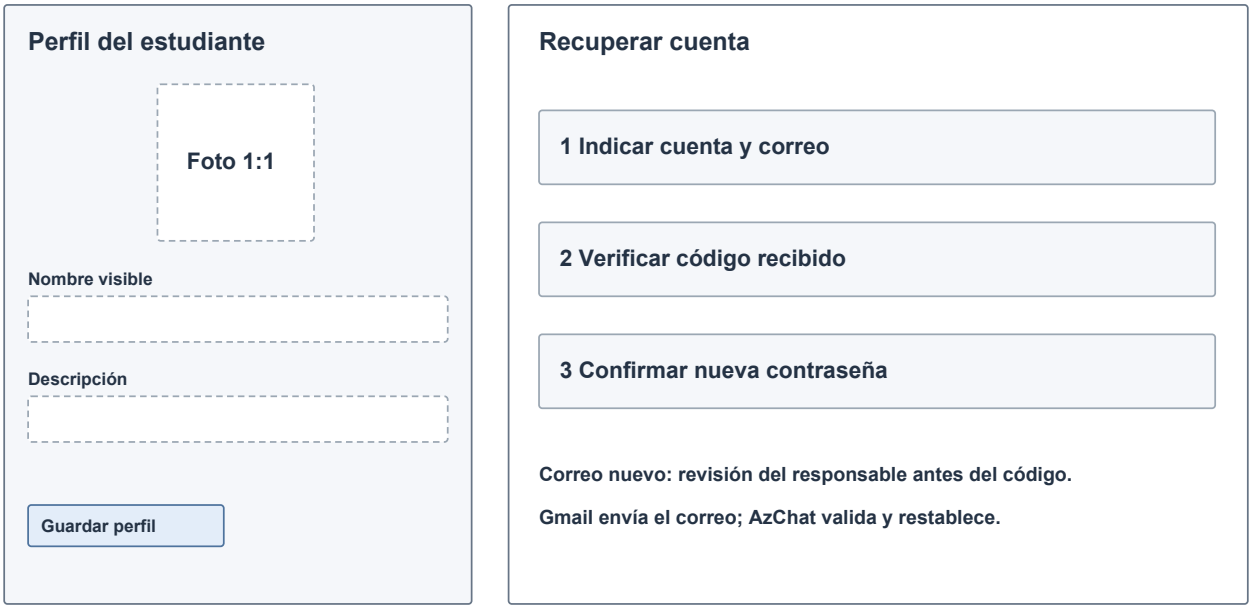

**Abrir chat → desplazar a conversación; Volver → recuperar el listado.**

**La barra inferior organiza las secciones. No se muestran dos paneles a la vez en el teléfono.**

Chats y conversación son vistas excluyentes; Perfil y Ajustes tienen su propia sección.

## 06 · Perfil y recuperación de acceso

AzChat · Diseño revisado · Elementos simplificados y datos ficticios




**La vista del perfil y el formulario de recuperación son recorridos independientes.**

Foto cuadrada y descripción; recuperación por código y revisión cuando corresponde.

## Explorar el boceto

Abrir boceto interactivo

Usa datos ficticios y conserva el chat privado simulado. Permite explorar también la disposición de Perfil y Ajustes. Los controles de muestra no envían archivos ni correos y no modifican cuentas reales.

## Recorrido de una coordinación académica

1. Ingresar y buscar al compañero o crear el grupo del equipo.
2. Abrir la conversación, escribir el avance y revisar el material antes de compartirlo.
3. Consultar respuestas e historial; volver a Chats para cambiar de conversación.
4. Usar Perfil y Ajustes para gestionar la cuenta; las indicaciones ! explican cada sección sin ocupar el historial.

## Imágenes y materiales del curso

La conversación tendrá controles separados para imágenes y archivos. Seleccionar, arrastrar o pegar una imagen abrirá la previsualización y el editor antes de subirla. El estudiante podrá recortar, ampliar, desplazar, girar, reflejar o ajustar el brillo; confirmar añadirá el resultado al pendiente y cancelar lo descartará.

Las imágenes enviadas mostrarán miniaturas. Pulsarlas abrirá un visor con zoom y un botón de descarga explícita. Abrir el visor no debe cambiar de página ni iniciar una descarga; Atrás o Escape lo cerrará conservando la conversación.

## Notas de voz para los compañeros

El micrófono pedirá permiso al pulsarlo. Detener la grabación dejará un audio pendiente para escucharlo antes de enviar; cancelar, salir del chat o pasar la página a segundo plano detendrá la captura. Se requerirá HTTPS o localhost y se ofrecerá adjuntar un archivo de audio como alternativa.

El reproductor integrado tendrá pausa, avance, duración, velocidad y descarga. La llegada de nuevos mensajes no debe reiniciar el audio que el estudiante esté escuchando.

## Perfil y nombres privados

Perfil permitirá cambiar foto, nombre y descripción. La foto se encuadrará en proporción cuadrada con arrastre, flechas y zoom antes de guardarse. Las fotos de otros compañeros podrán abrirse en el visor.

La información de un contacto permitirá asignarle un nombre privado. Ese nombre se usará en la lista, la cabecera y la identificación del autor para quien lo guardó; no alterará el perfil del compañero ni la vista de otros estudiantes.

## Grupos y preferencias del estudiante

La información del grupo reunirá nombre, descripción, foto, participantes y roles. Los cambios administrativos se mostrarán solo a quien tenga ese rol; salir exigirá dejar otro administrador cuando queden miembros.

En escritorio, Ajustes podrá permanecer como columna junto al chat; en celular tendrá su propia sección. Agrupará apariencia clara, oscura o del sistema, Enter para enviar, mensajes compactos, sonido, notificaciones, instalación, exportación y seguridad. Las preferencias de cuenta se conservarán; los permisos de micrófono, sonido y push dependen también del dispositivo.

## Recuperación y confirmaciones

El ingreso ofrecerá recuperar la contraseña. El estudiante revisará su cuenta y el correo propuesto antes de confirmar. La interfaz distinguirá espera de aprobación, código enviado, código vencido, rechazo y cambio autorizado; mostrar un mensaje de aprobación no debe anunciar que la contraseña ya cambió.

Registro y Ajustes permitirán un correo opcional con verificación. Los campos de contraseña tendrán mostrar u ocultar y confirmación cuando corresponda. La ayuda ! explicará el paso; los errores y las consecuencias de aprobar o cambiar una clave permanecerán visibles. El circuito interno se detalla en P9.

## Exportación e instalación

El estudiante podrá exportar todos sus chats o una selección desde Ajustes, y un chat desde su información. El TXT incluirá autores, fechas, referencias de respuesta y nombres de archivos, sin incorporar los binarios ni el contenido de mensajes eliminados.

Añadir a la pantalla de inicio ofrecerá instalación cuando el navegador lo permita o indicaciones para hacerlo. Los avisos requerirán permiso explícito y sesión vigente. Silenciar un chat también silenciará su aviso push; las notificaciones no mostrarán el texto privado del mensaje.

## Boceto de las herramientas de intercambio




Ejemplo ficticio: los materiales y las notas de voz comparten el historial; el perfil y el nombre privado se gestionan sin abandonar el contexto del chat.

# P11 · La información que se conserva

Qué se guarda para volver a consultar las conversaciones.

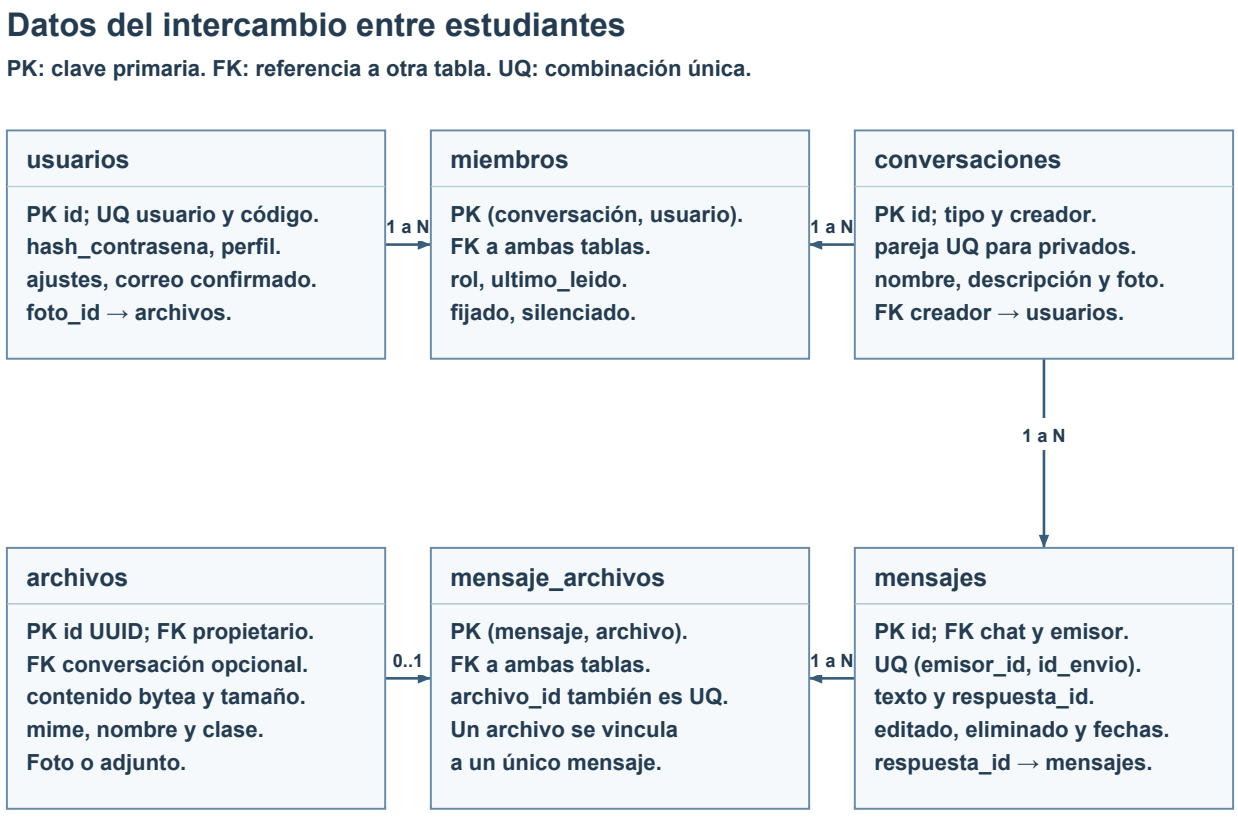
## Qué información se conserva

El modelo conserva la identidad de los estudiantes y el contexto de su intercomunicación: conversaciones, participantes, mensajes y materiales. Separa esos datos de las sesiones temporales y de los permisos de recuperación.

| Tabla | Campos y finalidad |
|---|---|
| usuarios | id, nombre_completo, codigo_estudiante y nombre_usuario únicos; hash_contrasena, descripcion, foto_id, ajustes JSON, ultima_actividad, correo_recuperacion y correo_confirmado_en. |
| conversaciones | id, tipo, nombre, descripcion, pareja única para chats privados, creador_id, foto_id y fecha_creacion. |
| miembros | PK compuesta por conversacion_id y usuario_id. Guarda rol, ultimo_leido, fijado, silenciado y fecha_ingreso. |
| mensajes | id, conversacion_id, emisor_id, id_envio UUID, texto, respuesta_id, fecha_hora, editado, eliminado y actualizado_en. |
| archivos | id UUID, usuario_id, conversacion_id opcional, nombre, mime, contenido bytea, tamano, clase y fecha. |

| Tabla | Campos y finalidad |
|---|---|
| mensaje_archivos | PK mensaje_id y archivo_id; archivo_id además es único para impedir vincular el mismo binario a dos mensajes. |
| sesiones | sid, sess JSON y expire. El JSON contiene identidad, vencimiento y estado temporal de los flujos de acceso. |
| notificaciones_push | id, usuario_id, sesion_id, endpoint único, p256dh, auth y actualizado_en. |
| contactos_privados | PK usuario_id y contacto_id; nombre privado para esa pareja propietario/contacto. |
| solicitudes_recuperacion | id, usuario_id opcional, sesion_hash, correo, version_clave, token_admin_hash, estado y fechas de creación, vencimiento y decisión. |
| correo_desafios | id, usuario_id y solicitud_id opcionales, sesion_hash, correo, proposito, version_clave, codigo_hash, intentos y fechas de creación, envío, vencimiento y consumo. |

## Relaciones del intercambio

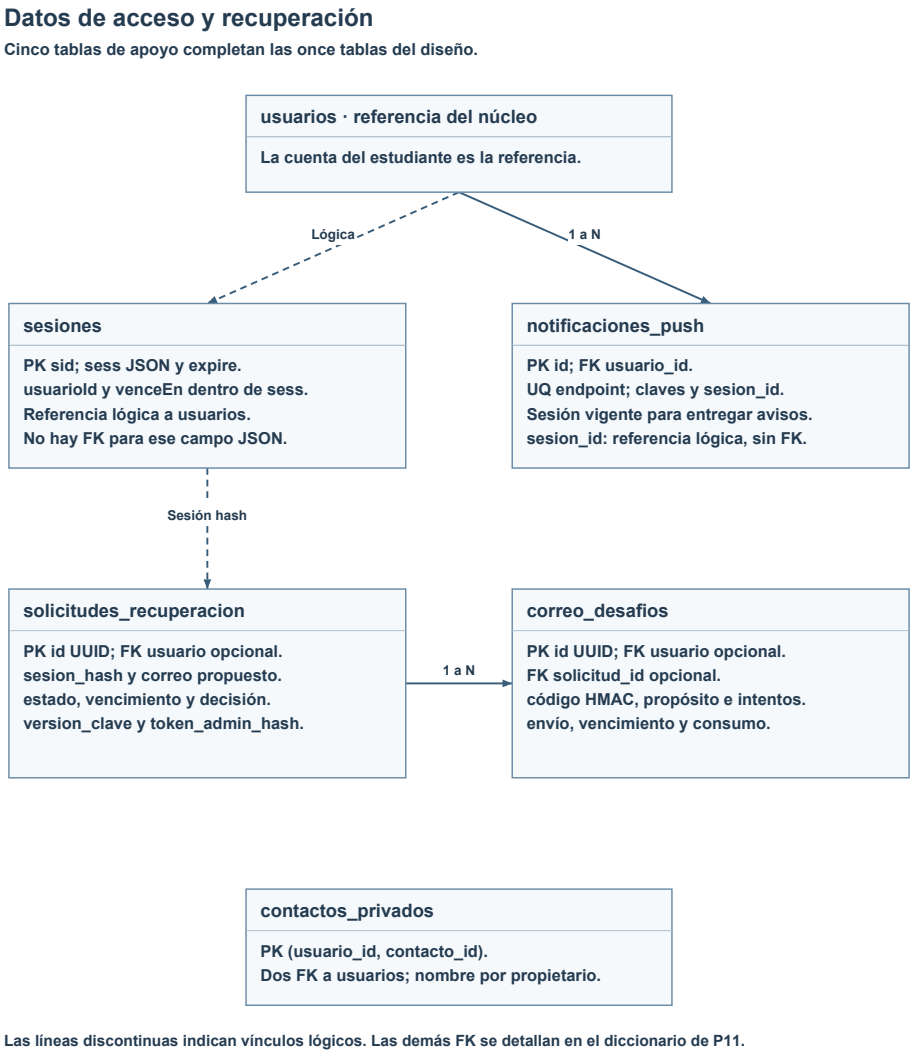



**Relaciones principales de 6 tablas. El diccionario detalla autoría, fotos y respuestas a otros mensajes.**
**Las 5 tablas de acceso, preferencias privadas y recuperación se muestran en la figura siguiente.**

Una relación muchos a muchos entre estudiantes y chats se resuelve mediante miembros. Mensaje y adjunto se enlazan con mensaje_archivos.

Un usuario puede participar en varias conversaciones. Cada mensaje tiene un chat y un autor; una respuesta referencia otro mensaje. El servidor exige que la respuesta y los adjuntos pertenezcan al mismo chat. Las fotos de usuarios y conversaciones también referencian archivos; su conversacion_id puede ser nulo.

## Relaciones de acceso y recuperación




Sesiones referencia al usuario dentro de JSON y no mediante una FK. Los vínculos por hash y sesion_id son lógicos; el servidor los comprueba.

contactos_privados tiene dos claves foráneas a usuarios: una para el propietario y otra para el compañero. Las tablas de recuperación pueden tener usuario nulo; no debe interpretarse una solicitud por sí sola como una cuenta acreditada. correo_desafios puede enlazar una solicitud de recuperación.

## Integridad e índices

- UNIQUE(emisor_id, id_envio) impide dos registros para el mismo intento; pareja evita duplicar un chat privado. La API construye la pareja con los identificadores ordenados.
- PRIMARY KEY(conversacion_id, usuario_id) evita pertenencias duplicadas. La administración del grupo bloquea la conversación para coordinar cambios simultáneos.

- El índice de mensajes por conversación e ID sirve al historial; el de conversación y actualización sirve a los cambios. Miembros se indexa por usuario y conversación.
- Los desafíos se indexan por cuenta, correo y hash de sesión para comprobar cuotas y vencimientos. Las sesiones se indexan por vencimiento y por la identidad del JSON.

### Conservación y privacidad

Cerrar una sesión no borra el historial. Eliminar un mensaje vacía su texto y activa la marca de eliminación; no equivale a un borrado físico de todos los archivos. Las subidas almacenadas siguen contando para la cuota. Quitar a un miembro corta su acceso; si se le añade de nuevo, podrá consultar el historial de ese grupo.

La contraseña se representa mediante un hash Argon2id; los códigos y permisos tienen vencimiento y consumo. Correos, hashes, códigos, secretos de push y nombres privados no forman parte del perfil público de un compañero.

# P12 · Construcción y validación

Cómo se construyó AzChat por incrementos, qué tecnologías intervienen y cómo se revisaron las mejoras.

### De la planificación a la construcción

AzChat nació para facilitar la intercomunicación entre estudiantes de la Universidad Nacional del Altiplano. Los pasos anteriores definen el problema, los usuarios, las funciones, los recorridos, la arquitectura y los datos. Este paso explica cómo esas decisiones se llevaron a una aplicación funcional y cómo el uso del sistema originó ampliaciones.

El relato reúne las etapas del trabajo realizado. Los incrementos siguientes agrupan cambios relacionados para facilitar su comprensión; no representan sprints con una duración fija ni un calendario que se haya registrado de antemano. La referencia técnica de esta revisión es Complete 2.9.0, con fecha documental 6 de noviembre de 2026.

### Metodología aplicada al trabajo

Para la construcción de **AzChat** se empleó un modelo de desarrollo incremental e iterativo, con un enfoque adaptativo.

El proyecto comenzó con una planificación inicial que permitió definir las funcionalidades principales del sistema y desarrollar una primera versión funcional.

Posteriormente, durante el proceso de desarrollo, se identificaron nuevas necesidades y oportunidades de mejora. Cada una de ellas fue analizada y planificada antes de proceder con su implementación e integración al sistema existente.

Este procedimiento permitió ampliar progresivamente las capacidades de AzChat, adaptando el desarrollo a los nuevos requisitos sin necesidad de reconstruir completamente la aplicación.

**Incremental** significa añadir capacidades sobre una base que funciona: después del chat se incorporaron grupos, archivos y audio. **Iterativo** significa revisar una capacidad ya construida: la navegación móvil y los ajustes cambiaron después de observar dificultades de uso. El **enfoque adaptativo** permitió priorizar nuevas necesidades y elegir una solución compatible con lo existente.

En cada mejora se siguió un ciclo: identificar la necesidad, revisar el alcance y las partes afectadas, decidir el cambio, implementarlo, comprobarlo e integrarlo. Si la comprobación revelaba un problema, se corregía antes de continuar. Se mantuvo una orientación ágil de entregas pequeñas y retroalimentación; no se atribuye al proyecto la aplicación formal de Scrum ni ceremonias que no se documentaron.




Cada incremento añade una capacidad; la revisión puede devolver el trabajo al análisis y al ajuste.

## Incrementos e iteraciones de la aplicación

### 1. Definir el alcance y probar el intercambio básico

La primera necesidad fue demostrar que dos estudiantes podían identificarse, conversar y volver a consultar los mensajes. Se organizaron los requisitos y los modelos de los pasos 1-11, y se construyó una primera versión funcional en la carpeta Web.

Esa versión permitió comprender la comunicación entre navegador y servidor. Al ampliar el producto se conservó Web como antecedente independiente y se trabajó en Complete. Así se pudo evolucionar sin perder la demostración inicial ni confundirla con la versión ampliada.

### 2. Ampliar la base de cuentas y conversaciones

El intercambio entre dos personas resultó insuficiente para coordinar equipos. Se añadieron conversaciones privadas, grupos, participantes y permisos. El modelo de datos pasó a distinguir la identidad del usuario, la conversación, su pertenencia y los mensajes.

Se preparó otro proyecto de Supabase para Complete y se realizó la copia hacia ese destino, conservando la base original. El servidor usa PostgreSQL con un rol limitado y verifica quién pertenece a cada chat antes de consultar o modificar información. Esta separación permitió probar el sistema ampliado sin convertir la base inicial en el entorno de experimentación.

### 3. Construir una interfaz para computadora y celular

Se incorporaron el logo de AzChat, los temas claro y oscuro, el perfil y los ajustes. El acceso quedó en un panel reducido; después de ingresar, el chat ocupa la pantalla. Se exploró una adaptación a React, pero se decidió volver a HTML, CSS y JavaScript para conservar la implementación solicitada.

Los bocetos del paso 10 guiaron la distribución y fueron revisados conforme aparecieron necesidades. En computadora la barra puede ampliarse al acercar el puntero; se corrigió que permaneciera abierta involuntariamente. En celular se separó la lista de chats de la conversación y se incorporaron las secciones Chats, Perfil y Ajustes. Se revisaron el desplazamiento entre vistas y la ausencia de scroll horizontal.

### 4. Compartir materiales y organizar los grupos

Además de texto, los estudiantes necesitaban compartir archivos y trabajar en grupos. Se incorporaron adjuntos y administración de participantes, junto con perfiles con fotografía y descripción. También se prepararon conversaciones de ejemplo y dos grupos, uno general y Los 4 Mosqueteros, para recorrer el sistema con contenido.

El servidor relaciona cada archivo con su propietario y con el mensaje que lo utiliza. Los permisos y límites se comprueban también en el servidor. Los ejemplos ayudaron a revisar la presentación, pero no sustituyen una prueba de capacidad ni convierten conversaciones reales en datos de ensayo.

### 5. Publicar el sistema para acceder desde otras redes

La ejecución local permitió revisar el funcionamiento antes de compartirlo. Después se preparó la publicación para que los estudiantes pudieran usar AzChat desde cualquier lugar con Internet. El código de Complete se mantuvo en un repositorio privado de GitHub.

Render ejecuta el servidor Node.js y ofrece la dirección HTTPS de la aplicación. Supabase mantiene la base PostgreSQL independiente. Las contraseñas de conexión, el secreto de sesión y las credenciales externas se configuran como variables privadas del servidor. Se revisó el paquete de publicación para excluir archivos de configuración privados y se comprobó la aplicación publicada.

### 6. Adaptar la conversación al uso habitual del teléfono

El botón Atrás del teléfono inicialmente podía sacar al estudiante de la página. Se integró la History API del navegador para volver desde una conversación, Perfil o Ajustes al listado. Al estar ya en Chats, la navegación permite salir normalmente.

Se añadió grabación con MediaRecorder y reproducción dentro del mensaje, con pausa, avance y velocidad. El micrófono se solicita cuando la persona pulsa el control. La presencia se obtiene con señales periódicas de las sesiones visibles: permite mostrar En línea o la última conexión conocida, sin afirmar que el estado sea instantáneo.

### 7. Mantener el acceso e incorporar avisos

Se amplió la duración de la sesión a 36 horas desde el ingreso. La sesión se guarda en PostgreSQL y una cookie identifica al navegador; cerrar sesión o restablecer la contraseña puede terminar ese acceso antes del vencimiento.

Se incorporaron un manifiesto web para añadir el acceso a la pantalla de inicio y un service worker para avisos push. El permiso de notificación se solicita explícitamente. En computadora se añadió un sonido

breve para los mensajes nuevos, con una preferencia para desactivarlo. Estas capacidades dependen de los permisos y de la compatibilidad del dispositivo; instalar la web no hace que las conversaciones funcionen sin conexión.

**8. Revisar la escritura y el contenido multimedia**

La revisión de los formularios llevó a añadir mostrar u ocultar la contraseña y confirmarla al crearla o cambiarla. La foto se ajustó mediante encuadre 1:1. Se añadió exportación TXT de un chat, varios o todos los permitidos y se mejoró el reproductor de audio.

Las fotografías de perfil y las imágenes compartidas se abren en un visor. La descarga requiere una acción explícita. Se separaron imágenes y archivos y se incorporó edición previa con recorte, giro, reflejo, zoom y brillo. Cancelar el editor no envía la imagen. Estas mejoras reutilizaron el sistema de archivos y sus permisos, sin rehacer el chat.

**9. Corregir la distribución y reducir interrupciones**

Los ajustes se organizaron en secciones desplegables. Tras revisar su uso, dejaron de ser una ventana emergente en computadora y pasaron a un panel entre la barra de chats y la conversación. El chat sigue disponible mientras se modifican preferencias.

Se corrigieron las barras de desplazamiento en campos con poco texto, manteniendo el scroll cuando el contenido sí supera el espacio. Se añadió un mosaico de Az y elementos informáticos para ambos temas. Las indicaciones extensas se reunieron en ayudas breves mediante el botón !. Son iteraciones sobre la interfaz existente, guiadas por problemas observados.

**10. Incorporar recuperación de cuenta y correo verificado**

Se añadió un correo de recuperación opcional y su verificación. La recuperación distingue un correo ya confirmado de uno nuevo, que requiere revisión del responsable. También se incorporaron nombres privados para reconocer contactos sin modificar el perfil que ven otras personas.

El primer transporte de correo se preparó con Resend en modo de pruebas, limitado al destinatario autorizado. Después se integró Gmail API mediante Google Cloud y OAuth 2.0 para enviar desde el correo del responsable. El cambio se concentró en el módulo de correo: los códigos, la revisión administrativa y el restablecimiento siguieron siendo responsabilidad de AzChat. Esta sustitución muestra cómo el enfoque adaptativo permitió cambiar una integración sin reconstruir las conversaciones.

## Tecnologías y responsabilidad de cada parte

| Tecnología | Uso concreto en AzChat |
|---|---|
| HTML, CSS y JavaScript | HTML estructura las pantallas; CSS define distribución, temas y adaptación al tamaño; JavaScript controla formularios, navegación y actualización del contenido. Complete no utiliza React. |
| Node.js y Express | Ejecutan el servidor y sus rutas HTTP. Reciben peticiones, validan datos y permisos y coordinan las operaciones con PostgreSQL. |
| Fetch y consultas periódicas | El navegador solicita cambios por HTTP sin recargar la página. Complete consulta novedades aproximadamente cada 1,8 segundos en primer plano y 6 segundos en segundo plano; no utiliza Socket.IO para este mecanismo. |
| PostgreSQL en Supabase y pg | Almacenan cuentas, grupos, mensajes, archivos y relaciones. pg es el cliente que usa Node.js para consultar la base con parámetros. El navegador no recibe las credenciales de la base. |
| Argon2id y sesiones | Argon2id calcula el hash de las contraseñas. express-session y connect-pg-simple administran las sesiones persistidas en PostgreSQL, con cookies protegidas y vencimiento. |
| Canvas, MediaRecorder y HTMLAudioElement | Canvas permite encuadrar y editar imágenes; MediaRecorder graba desde el micrófono autorizado; el reproductor HTML reproduce el audio dentro del chat. |
| History API, manifiesto y service worker | Gestionan el regreso entre vistas, la instalación web y los eventos de notificación. web-push y VAPID se usan desde el servidor para enviar avisos a las suscripciones permitidas. |
| Google Cloud, OAuth 2.0 y Gmail API | Google Cloud configura el proyecto y la autorización; OAuth autoriza al servidor a enviar con la cuenta del responsable; Gmail API transporta los mensajes de verificación. |
| GitHub privado y Render | GitHub conserva el código de Complete con acceso restringido. Render lo despliega y ejecuta en una dirección HTTPS pública. |
| PGlite y Playwright con Chrome | Permiten pruebas aisladas de datos y recorridos del navegador. Se distinguen de las comprobaciones contra el servicio real y de una prueba en teléfonos físicos. |

## Cómo funciona la recuperación con Gmail

El estudiante no inicia sesión en AzChat con Google. Ingresar sigue requiriendo su usuario o código y la contraseña de AzChat. Google autoriza la cuenta que envía los correos, no reemplaza la autenticación del estudiante. El remitente configurado es el correo del responsable del proyecto, Antonio.

1. El responsable habilita Gmail API en un proyecto de Google Cloud y autoriza el permiso de envío mediante OAuth 2.0. Las credenciales y el token de renovación quedan en el entorno privado del servidor.
2. Cuando se registra o añade un correo, AzChat genera un código y prepara el mensaje. Su módulo de correo obtiene o renueva un token de acceso y solicita el envío a Gmail API.

3. La persona recibe el correo y escribe el código en AzChat. El servidor verifica su correspondencia, vencimiento e intentos; Gmail transporta el mensaje, pero no decide si el código es válido.
4. Si se olvidó la contraseña y se indica el correo ya confirmado, se pasa a la comprobación del código. Si se propone un correo nuevo sin sesión, la solicitud requiere revisión del responsable y confirmación explícita antes de permitir la verificación.
5. Después de verificar el código, se escribe y confirma la nueva contraseña. AzChat guarda su hash, consume el permiso de recuperación y revoca las sesiones anteriores. Aprobar una solicitud por sí solo no cambia la contraseña.

Los códigos son de cinco dígitos, duran 70 segundos y permiten hasta cinco intentos. La solicitud administrativa vence en 24 horas y el permiso de restablecimiento dura 10 minutos. El detalle de reglas está en P5. Si Gmail rechaza la operación, AzChat informa del fallo y no afirma que el correo se envió.

El registro de publicación del 5 de noviembre de 2026 identifica Gmail como transporte y registra una entrega verificada. También registra el proyecto OAuth en estado Prueba. Google establece una caducidad de siete días del token de renovación en ese estado para este permiso; debe renovarse la autorización o gestionar el estado del proyecto. Es una condición de mantenimiento del servicio, no una pérdida de los chats.

Referencias técnicas: envío de mensajes con Gmail API y autorización OAuth 2.0 y vencimiento de tokens.

## Cómo se integra una mejora sin rehacer el sistema

Por ejemplo, para editar una imagen antes de enviarla se identificó que el cambio pertenecía a la preparación del adjunto. Se añadió un editor en el navegador; al confirmar, su resultado se entrega al flujo de subida ya existente. Se revisaron cancelación, vista previa, permisos y tamaños. El almacenamiento de mensajes y las conversaciones pudieron conservarse.

La recuperación siguió la misma idea: se añadieron verificaciones y solicitudes al modelo de datos y rutas específicas al servidor. Cambiar Resend por Gmail afectó al transporte del correo, mientras el proceso de recuperación siguió usando sus reglas. Cada modificación se integró después de comprobar las funciones nuevas y los recorridos que podía afectar.

## Validación y evidencia disponible

La construcción incluyó pruebas de API, recorridos de interfaz en Chrome, bases temporales con PGlite y comprobaciones del servicio HTTPS publicado. Los registros del proyecto describen pruebas de permisos por conversación, sesiones, grupos, archivos, exportación, navegación móvil, edición de imágenes y recuperación. Este documento explica esas evidencias previas; su actualización editorial no representa una nueva ejecución de todas las pruebas.

| Tipo de comprobación | Qué permite concluir y qué no |
| --- | --- |
| API y datos aislados | Comprueba reglas y resultados con cuentas de prueba: acceso ajeno rechazado, códigos vencidos, sesiones revocadas y permisos de archivos. No mide por sí sola el rendimiento en Internet. |

| Tipo de comprobación | Qué permite concluir y qué no |
|---|---|
| Interfaz en Chrome | Revisa controles, teclado, temas y anchos de pantalla; permite detectar paneles bloqueados y desbordamientos. La emulación móvil no equivale a probar todos los teléfonos. |
| Servicio publicado | Los registros de publicación comprueban HTTPS, salud, acceso y operaciones seleccionadas en el destino real. No garantizan disponibilidad permanente ni sustituyen un monitoreo. |
| Correo y notificaciones | Se registra entrega real de correo en la integración Gmail. Las pruebas simuladas de micrófono o push no acreditan todas las condiciones de hardware ni la recepción en cada proveedor. |

Quedan por evaluar de forma específica la carga concurrente prevista, una matriz más amplia de navegadores y teléfonos físicos y la recepción de notificaciones en esos dispositivos. Los criterios siguientes sirven para repetir la aceptación después de cambios importantes.

## Criterios de aceptación

| ID y relación | Escenario y resultado esperado |
|---|---|
| CA-01 · RF-01 y RF-02 | Registrar dos estudiantes distintos; rechazar datos inválidos y duplicados; ingresar por usuario o código y terminar el acceso al salir o vencer. |
| CA-02 · RF-04 y RF-05 | Dos estudiantes abren el mismo privado, incluso simultáneamente. En un grupo se aplican roles y no se retira al último administrador si quedan miembros. |
| CA-03 · RF-06 y RF-09 | A envía a B y a su grupo; cada destinatario autorizado recupera una sola copia. Persisten los mensajes al volver a ingresar y se distinguen envío y lectura. |
| CA-04 · RF-06 y RF-09 | Interrumpir antes y después del guardado. Reintentar el mismo intento devuelve un solo registro; reutilizarlo con otro contenido da conflicto. |
| CA-05 · RF-07 y RF-08 | Compartir un material y una nota de voz. Cancelar antes de subir no envía; otra cuenta ajena no descarga; un mensaje eliminado no entrega su adjunto. |
| CA-06 · RF-09 y RF-10 | Paginar y buscar un historial amplio sin duplicaciones ni omisiones. Exportar solo chats a los que pertenece el solicitante. |
| CA-07 · RF-12 | Verificar un correo, agotar intentos y comprobar vencimiento y consumo. Para correo nuevo exigir revisión; aprobar no cambia la clave. Restablecer revoca accesos previos. |
| CA-08 · RF-03, RF-11 y RF-13 | Perfil y ajustes conservan su valor. El nombre privado solo cambia la vista de su propietario. Los avisos dependen del permiso y la sesión del dispositivo. |
| CA-09 · RF-14 y RNF-03 | Completar la conversación con teclado y celular; abrir y cerrar ayudas, visor y ajustes; conservar foco y acceso al regresar. |
| CA-10 · RNF-01 y RNF-04 | La cuenta C intenta acceder al chat A-B por URL y por API: no obtiene historial ni archivos. Una sesión inválida no autoriza operaciones. |

## Escenario de capacidad

Se propone conectar 30 estudiantes en 15 parejas durante 2 minutos, con un mensaje por persona cada 10 segundos. Se evaluará que cada mensaje aparezca en un máximo de 3 segundos con servidor activo y conexión estable. Es un objetivo que deberá medirse, no un resultado de esta planificación.

La evidencia debe incluir hora de envío y aparición, mensajes esperados y recibidos, duplicados, omisiones, fallos y condiciones de red. Se registrará por separado cualquier arranque tras inactividad.

## Relación con los modelos anteriores

P4 resume los objetivos en un caso general y desarrolla los recorridos específicos. P7 explica el envío; P8 y P9 distribuyen las responsabilidades; P10 mantiene bocetos esquemáticos de la organización de pantalla; P11 representa la información persistente. P12 conecta esas decisiones con la construcción y la revisión. Los bocetos actualizados expresan el diseño consolidado y su evolución, no se presentan como evidencia de que todos los detalles existieran desde la primera planificación.